

**VALLURUPALLI NAGESWARA RAO VIGNANA JYOTHI INSTITUTE
OF ENGINEERING AND TECHNOLOGY**

AN AUTONOMOUS INSTITUTE

(Approved by AICTE, New Delhi, Govt of T.S and Affiliated to JNTU,
Hyderabad) Accredited by NBA and NAAC with 'A++' Grade

Bachupally,

Hyderabad – 500090

Telangana, India.

**DEPARTMENT OF ELECTRONICS AND COMMUNICATION
ENGINEERING**



SEVEN HABITS OF HIGHLY EFFECTIVE PEOPLE

1. Be Proactive.
2. Begin with the end in mind.
3. Put first things first.
4. Think Win-Win.
5. First Understand, then be understood.
6. Synergies.
7. Sharpen Your Saw.

We have followed the above 7 steps during the course of our project work

UPPALA BHARGAVA SAI

19071A04Q8

SHAIK SAJIDA

19071A04P6

GUTTA KARTHIKEYA

19071A04K4

**ERROR DETECTION AND CORRECTION USING
IMPROVED REDUNDANT MATRIX CODE**
**PROJECT WORK SUBMITTED IN PARTIAL FULFILLMENT OF THE
REQUIREMENT FOR THE AWARD OF THE DEGREE OF**
BACHELOR OF TECHNOLOGY
IN
ELECTRONICS AND COMMUNICATION ENGINEERING

Submitted by

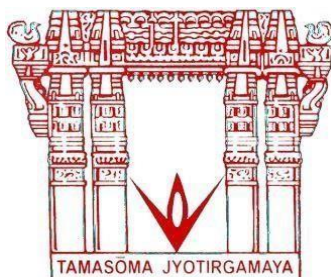
UPPALA BHARGAVA SAI	19071A04Q8
SHAIK SAJIDA	19071A04P6
GUTTA KARTHIKEYA	19071A04K4

UNDER THE GUIDANCE OF

Dr. G. SHANTHI

Assistant Professor

VNRVJIE



Department of Electronics and Communication Engineering

**VALLURUPALLI NAGESWARA RAO VIGNANA JYOTHI INSTITUTE
OF ENGINEERING AND TECHNOLOGY**

AN AUTONOMOUS INSTITUTE

(Approved by AICTE, New Delhi, Govt of T.S and Affiliated to JNTU,
Hyderabad) Accredited by NBA and NAAC with 'A++' Grade

Bachupally,

Hyderabad – 500090

Telangana, India.

2022-23

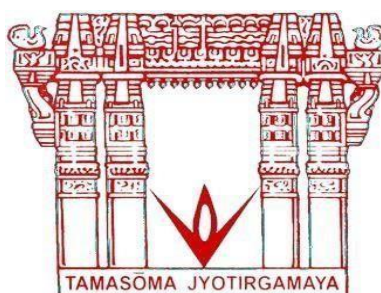
**VALLURUPALLI NAGESWARA RAO VIGNANA JYOTHI INSTITUTE
OF ENGINEERING AND TECHNOLOGY**

AN AUTONOMOUS INSTITUTE

(Approved by AICTE, New Delhi, Govt of T.S and Affiliated to JNTU,
Hyderabad) Accredited by NBA and NAAC with 'A++' Grade

Bachupally,
Hyderabad – 500090
Telangana, India.

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING



CERTIFICATE

This is to certify that the Major project report entitled “**Error detection and Correction using Improved Redundant Matrix Code**” being submitted by Uppala Bhargava Sai (19071A04Q8), Shaik Sajida (19071A04P6), Gutta Karthikeya (19071A04K4) in partial fulfilment of the degree of Bachelor of Technology in Electronics and Communication Engineering during the academic year 2022 –2023.

Certified further, to the best of our knowledge, the work reported here is not a part of any other project based on which a degree or an award has been given on an occasion to any other candidate. The result has been verified and found to be satisfactory.

SUPERVISOR

Dr. G. Shanthi
Assistant Professor
Department of ECE
VNRVJIET, Hyd.

HEAD OF THE DEPARTMENT

Dr. S. Rajendra Prasad
Professor and Head
Department of ECE
VNRVJIET, Hyd.

SIGNATURE OF THE EXTERNAL EXAMINER

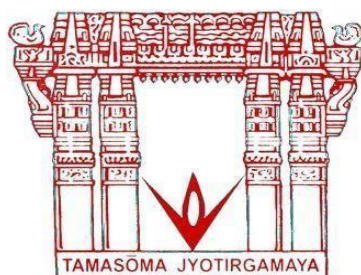
**VALLURUPALLI NAGESWARA RAO VIGNANA JYOTHI INSTITUTE
OF ENGINEERING AND TECHNOLOGY**

AN AUTONOMOUS INSTITUTE

(Approved by AICTE, New Delhi, Govt of T.S and Affiliated to JNTU,
Hyderabad) Accredited by NBA and NAAC with 'A++' Grade

Bachupally,
Hyderabad 500090
Telangana, India.

**DEPARTMENT OF ELECTRONICS AND COMMUNICATION
ENGINEERING**



DECLARATION

We do declare that the Major project report entitled **ERROR DETECTION AND CORRECTION USING IMPROVED REDUNDANT MATRIX CODE** submitted to the department of Electronics and Communication Engineering (ECE), Vallurupalli Nageswara Rao Vignana Jyothi Institute of Engineering and Technology, Hyderabad, in partial fulfilment of the requirement for the award of the degree of **BACHELOR OF TECHNOLOGY** in Electronics and Communication Engineering is the bona-fide record of the project report presented under the Supervision of **Dr. G. Shanthi**, Assistant Professor, VNRVJIET.

Also, we declare that the matter embodied in this project report has not been submitted by me in full or in any part there for the award of any degree/diploma of any other institution or university previously.

Place: Hyderabad

Date:

UPPALA BHARGAVA SAI

SHAIK SAJIDA

GUTTA KARTHIKEYA

ACKNOWLEDGEMENT

We would like to convey our heartfelt gratitude to **Dr. G. SHANTHI**, Assistant Professor, Electronics and Communication Engineering, for his invaluable assistance and unflinching encouragement during our project effort.

Our sincere thanks to **Dr. S. RAJENDRA PRASAD**, Professor and Head of Department of Electronics and Communication Engineering, VNRVJIET, Hyderabad for the encouragement and guidance provided during our project.

We are thankful to **Dr. C. D. NAIDU**, Principal, VNRVJIET, Hyderabad, for giving us permission to carry out this technical project.

We are grateful to all members of the ECE department at VNRVJIET for their assistance with this project.

Finally, we want to express our gratitude to our family and friends for their unwavering moral support.

UPPALA BHARGAVA SAI

SHAIK SAJIDA

GUTTA KARTHIKEYA

ABSTRACT

Mobile way communication over the Internet requires error detection and correction. Error Control describes how networks handle and detect errors at the data link layer. Each network connection is assumed to have two channels. One for data traffic and one for signaling and control. Robust error detection is encoded on the traffic channel based on the information bits and the correction code to form the transmitted data stream. Currently, the error is tolerated by the SRAM memory frame, so Single Cell Upsets (SCU) or Multiple Cell Upsets (MCU) can occur. One of the main reasons for the MCU is the astronomical radiation. The proposed technique is called Improved Redundant Matrix Code to protect SRAM-based memories from the detuning of a large number of components. The Improved Redundancy Matrix Code (IRMC) is used to detect and troubleshoot storage devices. IRMC's detection rate and adjustments are in contrast to other rescue codes. Space, power, and defer overhead is only. Results should prove that IRMC has higher quality and higher correctability for MCU tuned for the lower power and delay. Xilinx 14.7 is the tool used for the process and using Verilog HDL, programming of the algorithm is performed.

Keywords: Error Correction Code (ECC), Multiple Cell Upsets (MCU), Improved Redundant Matrix Code (IRMC), Multiple error detection, Multiple error correction Memory.

INDEX

CHAPTER 1 - INTRODUCTION	1-5
1.1 Related Work	2
1.2 Improved Redundant Matrix code algorithm	3-4
1.3 Problem Statement	4
1.4 Motivation	4
1.5 Objective	5
1.6 Tools Used	5
CHAPTER 2 - LITERATURE SURVEY	6-8
CHAPTER 3 – ENCODER OF IRMC ALGORITHM	9-11
CHAPTER 4 – DECODER OF IRMC ALGORITHM	12-15
CHAPTER 5 – WORKING OF IRMC ALGORITHM	16-17
CHAPTER 6 – RESULTS	18-24
6.1 Simulation of IRMC Algorithm using Verilog HDL Language for 32-bit input	18-19
6.2 Simulation of IRMC Algortihm using Verilog HDL language for 64-bit input	20-21
6.3 Comparison Results of 32-bit input	22-23
6.4 Comparison Results of 64-bit input	23-24
CHAPTER 7 – CONCLUSION AND FUTURE SCOPE	25
7.1 Conclusion	25
7.2 Future Scope	25
REFERENCES	26-27
APPENDIX – A: PUBLICATION STATUS & PAPER	28-38
APPENDIX – B: PLAGIARISM REPORT	39-44
APPENDIX – C: PROJECT TEAM	45

LIST OF FIGURES

Figure 3.1: Block Diagram of IRMC Encoder	10
Figure 3.2: Logical organization of original input and redundant bits	11
Figure 4.1: Block Diagram of IRMC Decoder	13
Figure 5.1: Error bit correction	17
Figure 6.1.1: Design report details of 32-bit input	18
Figure 6.1.2: Simulation result of 32-bit input	18
Figure 6.1.3: Design summary of 32-bit input	19
Figure 6.1.4: Timing Details of 32-bit input	19
Figure 6.2.1: Design report details of 64-bit	20
Figure 6.2.2: Simulation result of 64-bit	20
Figure 6.2.3: Design summary of 64-bit	21
Figure 6.2.4: Timing Details of 64-bit input	21
Figure 6.3.1: Graphical representation of delay analysis for 32-bit input	22
Figure 6.3.2: Graphical representation of power analysis for 32-bit input	23
Figure 6.4.1: Graphical representation of power analysis for 64-bit input	24
Figure 6.4.2: Graphical representation of delay analysis for 64-bit input	24

LIST OF TABLES

Table 6.3.1: Comparison of delay and power among existing algorithms & proposed algorithm (IRMC)	22
Table 6.4.1: Comparison of delay and power among existing algorithms & proposed algorithm (IRMC)	23

LIST OF ABBREVIATIONS

IRMC – Improved Redundant Matrix Code

OLS – Orthogonal Latin Square

DMC – Decimal Matrix Code

CHAPTER 1

INTRODUCTION

All error-detection and correction schemes add some redundancy (i.e., some extra data) to a message, which receivers can use to check consistency of the delivered message, and to recover data that has been determined to be corrupted. Error-detection and correction schemes can be either systematic or non-systematic. In a systematic scheme, the transmitter sends the original data, and attaches a fixed number of check bits (or parity data), which are derived from the data bits by some deterministic algorithm. If only error detection is required, a receiver can simply apply the same algorithm to the received data bits and compare its output with the received check bits; if the values do not match, an error has occurred at some point during the transmission. In a system that uses a non-systematic code, the original message is transformed into an encoded message carrying the same information and that has at least as many bits as the original message.

Systems for transportation and communication are two that provide services to consumers. Clients can quickly assemble data for these services through parallel transmission. During transmission, all data bits are sent simultaneously through different communication connections [1]. Here, a direct link is established between the number of active lines and the number of bits to send. Each group of bits is sent over the data lines on each clock pulse. Practically speaking, parallel transmission requires "N" communication channels to work. Contrary to popular belief, sequential transmission requires only one communication channel, which reduces the total cost compared to serial transmission [2]. Memories will be solid only if the information stored inside is error-free. If there is an error, you will know at once.

Communication difficulties that arise during strategy implementation are caused by transmission errors. Soft errors are those caused by external noise or radiation rather than design or manufacturing defects [3]. To communicate on the phone using the web, error detection and correction codes are required. Packages are received by the device through a port in the framework programs, processed with it, and then transmitted to one or more output ports. Data in memory is checked while this handling is in progress [4]. To detect and correct errors, the stored data needs to be examined. The basic idea is to incorporate revision to correct errors and add information to the message that receivers can use to evaluate the relevance of the message.

When using a non-random code, the encoded message is modified to have a certain number

of bits that are less likely. Another grouping is a type of systematic module, in which the transmitter sends unique data with a predetermined number of parity bits, such as check bits derived from information bits [5]. When only error detection is required, the beneficiary uses the same computation to compare the received information bits with the received check bits. If the attributes do not match, we infer that an error occurred at some point during the transmission process. Typically, error-correcting codes are used for lower-layer communication, equivalent to reliable limits in media such as CDs, DVDs, hard disks, and RAM [6-7].

1.1 RELATED WORK

A disadvantage of many error-detection algorithms such as CRC is that they can only detect errors. Once errors in the data are detected at the destination, they are impossible to correct, and a new transmission of the data is required to receive the message [8]. Error-detection and correction algorithms have significant weaknesses in managing designs with low power and delay characteristics and eliminating redundancy. It is also important to guarantee decoder hardware around memory blocks that have become susceptible to soft errors [9]. It is used to identify and characterize a distinct class of error-correcting codes whose redundancy makes the setup of fault-secure detectors (FSD) particularly critical. In a situation where the structural failure rate is determined by the encoder and decoder failure rates, it also accounts for the importance of ensuring the encoder and decoder hardware against transient errors [10-11]. It uses Parity Check Matrix and Matrix Generator to encode the data and encode it by matrix multiplication and addition [12]. Parity matrix codes are designed to improve error correction capabilities with extra bits. This fix contains minor performance improvements and fixes most memory errors.

Many real-time applications like, Internet calls require error detection and correction. The term "error control" refers to the management and detection of errors in the data link layer of the network [13]. This is a common setting for using error correction code (ECC) in memory devices. The encoding/decoding circuit of ECCs used for data storage must be efficient in terms of bandwidth, power and delay [14]. The algorithm used in this study is called improved redundancy matrix coding (IRMC). IRMC is used to detect and repair defects in memory devices [15-16]. The range, power, and delay of other correction codes are comparable to IRMC's detection and correction speed [1]. The Improved redundancy matrix code (IRMC) algorithm is a coding technique used in Large Scale Integration (VLSI) technology to improve the reliability of digital circuits. This is a type of error correction code (ECC) that allows you to detect and correct errors that may occur during data transfer or storage [2].

1.2 IMPROVED REDUNDANT MATRIX CODE ALGORITHM

The improved redundancy matrix code algorithm is based on the standard redundancy matrix code, but includes some additions to improve performance. One of the main improvements is to use multiple redundancy matrices instead of a single redundancy matrix. This approach allows algorithms to detect and correct errors more efficiently, resulting in improved data.

Another improvement is to use an optimized matrix to encode the data. The optimized matrix is designed to reduce the number of possible errors during transmission. This reduces the need for error correction, which in turn increases data transfer speeds.

The algorithm also includes a dynamic configuration feature that allows the matrix to be configured in real-time based on detected error patterns. This helps improve error correction performance and allows the algorithm to adapt to changing transition conditions.

Finally, the improved redundancy matrix code algorithm includes a comprehensive error correction mechanism that can correct multiple errors simultaneously. This further improves the efficiency and accuracy of error correction.

In the IRMC algorithm, a redundant bit matrix is added to the original data to generate a new code word [1]. This redundant bit matrix is carefully designed to ensure that any bit errors are detected and corrected, and multiple bit errors are detected.

The IRMC algorithm is particularly useful in VLSI technology where the circuit density is very high and the possibility of errors is high. The use of IRM code algorithm in VLSI can improve circuit reliability, reduce the risk of data corruption and improve overall system performance [1].

In addition, the IRMC algorithm is also used in other fields such as telecommunications, data storage, and satellite communications. This is a widely accepted method of error correction and its use is expected to increase as technology advances.

In the proposed work, the IRMC is executed for error detection and correction of 32-bit and 64-bit input codes using Verilog in Xilinx tool in the below mentioned stream of steps:

- i. Taking the Original 32-bit/ 64-bit as input for Encoder of IRMC algorithm.
- ii. Generating Horizontal Redundant bits using AXL calculation and Vertical Redundant bits using Logical Organization of Original input in Encoder region. Taking Erroneous 32-bit/64-bit as input for decoder part.
- iii. Performing step (ii) for the erroneous input to get respective horizontal and vertical redundant bits.
- iv. Generating Syndrome value to detect and correct the error, if error exists.
- v. Using the generated syndrome value, the error can be detected and corrected, that resulting into the erroneous input bit will be regenerated, which should be same as that of original input bit.

1.3 PROBLEM STATEMENT

- One disadvantage of several error-detecting algorithms, such as CRC, is that they can only detect errors. Once faults in the data are discovered at the destination, they are impossible to fix, and a new transmission of the data is required to receive the message.
- Managing the design with less power and delay characteristics, as well as decreasing redundancy, are important weaknesses of mistake-detecting and repairing algorithms.

1.4 MOTIVATION

In order to communicate over the phone using the internet, error detection and correction are required. The term "error control" refers to the way in which the network manages and recognizes mistakes at the data connection layer. The use of Error Correction Codes (ECC) in memory devices is a common setup. The encoding/decoding circuits of ECCs used for information storage must be effective in terms of territory, power, and deferral. The algorithm utilized in this research is called Improved Redundant Matrix Code (IRMC). IRMC is used to identify and fix defects in memory devices. Other redress codes' region, power, and postponement overheads are compared with the IRMC's rate of discovery and adjustment.

1.5 OBJECTIVE

- Encoding process of 32-bit/64-bit inputs.
- Decoding process of the erroneous 32-bit/64-bit input.
- Working of IRMC algorithm.
- Message code simulation results analysis using the Xilinx tool
- Decimal Matrix code, Bit Manipulation method and the proposed Improved Redundant Matrix Code (IRMC) are the algorithms that can be compared for delay and power.

1.6 TOOLS USED

- Xilinx 14.7 ISE

CHAPTER 2

LITERATURE SURVEY

S. No	Name of the journal	Name of the authors	Title of the paper	Key Findings
1	In IEEE Access	R. Jenila, C. Supraja, C. Kammani Pappa, N, Dharani	“VLSI Implementation of Error Detection and Correction Codes for Space Engineering,”	To recover the data, again XOR operation was performed between the encoded bits and the recalculated encoded bits. After analyzing, verification, selection and correction process takes place.
2	ICTACT Journal on Microelectronics	V. Bhanumathi and M. Sunandini	“VLSI Architecture for Error Detection and Correction based on XOR,”	As technology is scaled down, MCUs become a more problematic one in SRAM memory, because MCUs flip the logical state in memory thereby affecting its reliability by introducing errors.
3	Research Gate	Bao Liu	“Error-detection/correcting-code-based self-checked/corrected/time d/circuits	Propose a group of error detecting/correcting-code (EDC/ECC)- based self-checked/corrected/timed circuits for logic robustness and performance scalability in nanoscale VLSI design.
4	In ICRTCRET Journal	K. Kaviya Priya, Mr. M. Palaniappan	“Implementation of Error Detection and Correction Codes using VLSI,”	It is processed by adding an extra bit to each character. The bit is obstinate by a number of factors such as the type of parity and the data character has the number of logic-one bits.

Error Detection and Correction using Improved Redundant Matrix Code

5	In IEEE Transactions on Very Large-Scale Integration (VLSI) Systems, vol. 27	Rajeev Kumar Abhishek Gupta	“VLSI Implementation of Block Error Correction Coding Techniques,”	For error correction and detection, several approaches are used. In information theory and coding techniques, error detection techniques provide a way for reliable communication instead of unreliable communication.
6	The International journal of analytical and experimental modal analysis	Poralla Divya U. Gnaneshwara Chary	“Design and Implementation of VLSI and Architecture for Error Correction and Detection,”	The transmitter sends the original data, and attaches a fixed number of parity bits, which are derived from the data bits by xor operation between three data bits to get one parity bit.
7	IEEE Transactions on Computers, Vol. XX	Hamed Farbeh, Hyeonggyu Kim	“Maximizing Efficiency of Error Detection/Correction Codes in Associative Cache Memories”	When configured to maximize the protection capability, EUC provides a fourfold higher burst error detection/correction capability. Moreover, EUC is orthogonal to previous protection schemes and they can be redesigned based on the EUC architecture to further improve their efficiency.
8	IEEE Access	Luis-J. Saiz-Adalid, Joaquin Gracia-Moran, J-Carlos Baraza-Calvo	“Ultrafast Codes from Multiple Adjacent Error Correction and Double Error Detection”	ECC introduces encoding and decoding latencies, silicon area usage, and power consumption. In other computer units, these parameters should be optimized, and redundancy would be less important.

Error Detection and Correction using Improved Redundant Matrix Code

9	In IEEE Transactions on Electromagnetic Compatibility	Jonas Van Waes, Jens Vankeirsbilck, Jonas Lannoo	“Resilience of Error Correction Codes against Harsh Electromagnetic Disturbances: Fault Elimination for Triplication-Based Error Correction Codes”	It is found that a code with inversion is significantly more robust to those disturbances. Furthermore, systematic fault injection is also performed to search for vulnerabilities within the code itself.
---	---	--	--	--

CHAPTER 3

ENCODER OF IRMC ALGORITHM

Using the Improved/ Enhanced Redundancy Matrix Coding Algorithm, the error detection and correction encoder code involves adding additional bits to the original message to detect and correct errors that may occur during transmission or storage. This process involves creating a redundancy matrix code that can be used to detect errors and locate them in messages [1].

The Enhanced Redundancy Matrix Coding algorithm is an error detection and correction technique that can detect and correct multiple errors in a message. It works by creating a parity check matrix that is used to detect errors in received messages. The same matrix is created by adding additional rows and columns to the original message matrix [1]. These additional rows and columns contain parity bits used to check the integrity of the message.

To encode a message using the enhanced redundancy matrix encoding algorithm, the following steps are usually performed:

- i. The message is converted into binary code.
- ii. Binary messages are divided into blocks of equal size.
- iii. The redundancy matrix is created by adding additional rows and columns to the original message matrix. These additional rows and columns contain parity bits used to check the integrity of the message.
- iv. The message matrix and the redundancy matrix are multiplied together to produce the encoded message.
- v. Coded messages are sent or saved.

If an error occurs during transmission or storage, the recipient can use the parity check matrix to determine the location and type of error. If the error is a single error, it can be corrected using the parity bit. If some errors are detected, the message may need to be sent. Overall, the error detection and correction code encoder using the improved or enhanced redundancy matrix code algorithm provides an efficient method for detecting and correcting errors in messages. It is widely used in various applications such as communication systems, data storage systems, and digital media [1].

The Block diagram of Encoding part of IRMC is described below:

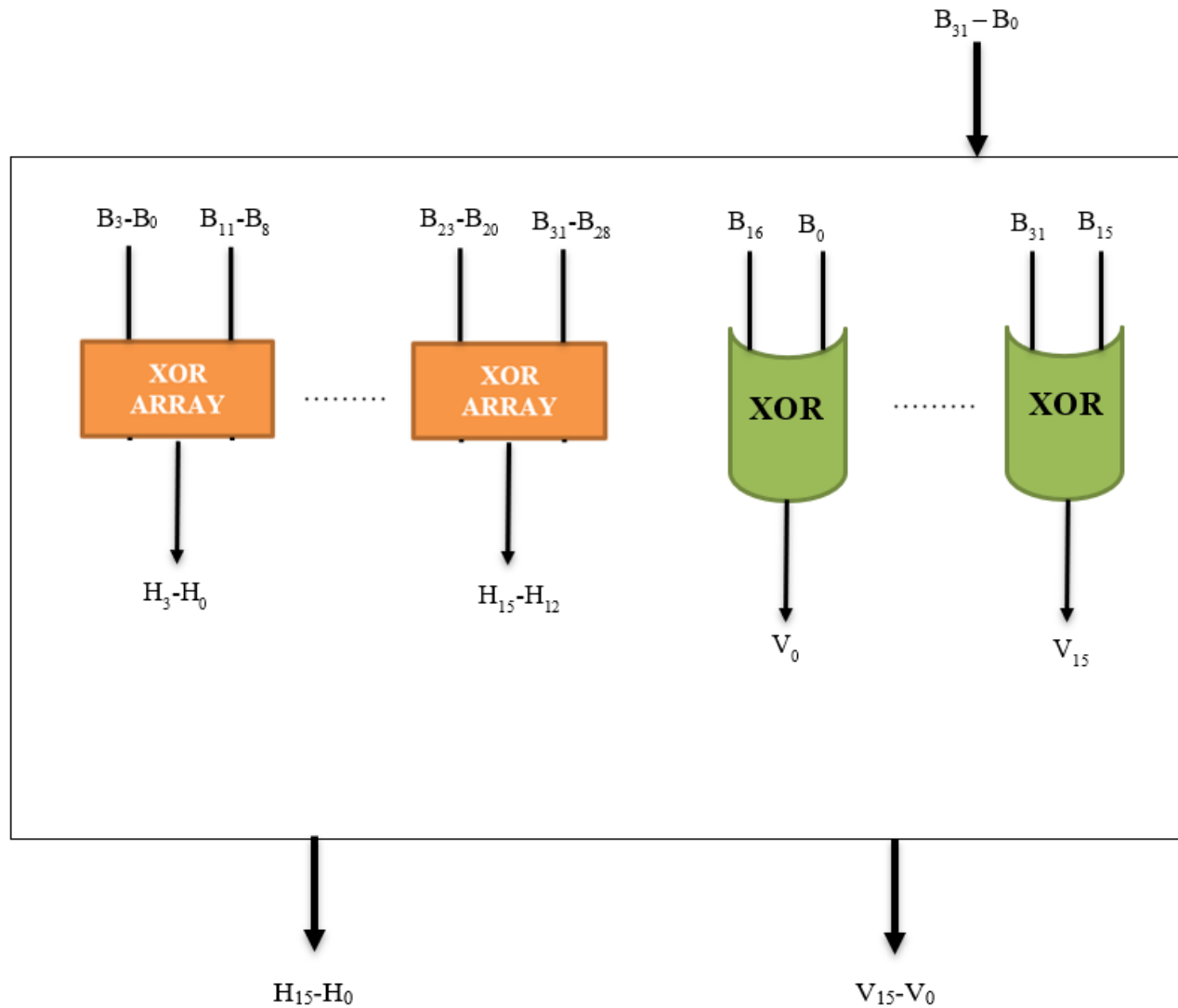


Figure 3.1: Block Diagram of IRMC Encoder

Consider, an encoder was introduced that registers repeating bits using logic arrays and XOR inputs. Additional parts are added to distinguish or correct possible defects [1]. The figure 1 shows the horizontal redundant bits ($H_{15}-H_0$) and the vertical redundant bits ($V_{15}-V_0$) generated for the 32-bit input ($B_{31}-B_0$). The algorithm in the IRMC encoder starts with the same matrix. Choosing the logic conditions to calculate each equation and syndrome is not difficult when the equation matrix is chosen. For example, the AXL calculation uses symbols S_0 ($B_3B_2B_1B_0$) and S_2 ($B_{11}B_{10}B_9B_8$) to obtain the extra bits [$H_3H_2H_1H_0$].

In the error detection and correction code encoder using the Improved/Enhanced Redundant Matrix Coding Algorithm, AXL refers to additional checks for functions such as XOR. A method used to calculate additional redundancy that can be used for error detection and correction.

Error Detection and Correction using Improved Redundant Matrix Code

The AXL computing process involves XORing different parts of the input data bits and then combining the results to create additional redundant bits. Specifically, for a given data set, the encoder selects a set of indices that correspond to certain bits in the block, and then XORs the integer values together to produce a single checksum. This process is repeated for the majority of bits, and the resulting check values are summed and concatenated to produce the final AXL bit.

AXL bits are then added to the original data bits to create a new, larger data set that can be improved with error detection and correction capabilities. In the decoder, the AXL bit is used together with other redundant bits to calculate syntax, which is then used to detect and correct errors that may occur during transmission.

In general, the AXL algorithm is an important component of the improved redundancy matrix code algorithm because it provides error detection and correction capabilities that can improve the accuracy and reliability of data transmission.

Figure 3.2 shows the Information redundancy and logical organization of original input (for 32-bit) and generated horizontal and vertical redundant bits:

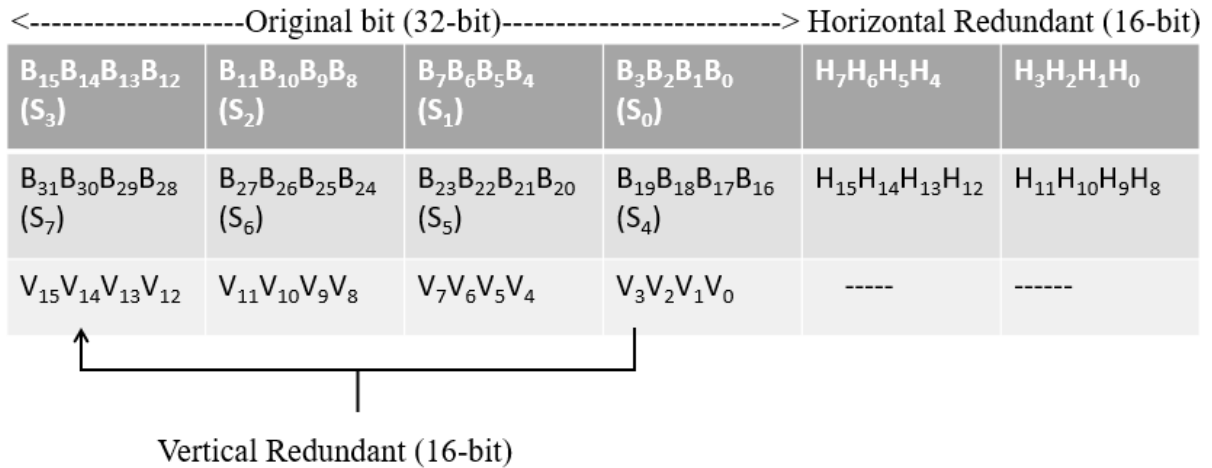


Figure 3.2: Logical organization of original input and redundant bits

The horizontal and vertical redundant bits are computed in the below manner:

Horizontal Bits generation using AXL calculation,

$$B_{11}B_{10}B_9B_8 \wedge B_3B_2B_1B_0 = H_3H_2H_1H_0$$

$$B_{15}B_{14}B_{13}B_{12} \wedge B_7B_6B_5B_4 = H_7H_6H_5H_4 \quad (1)$$

Vertical Bits generation,

$$V_n = B_n \wedge B_{n+16} \quad (2)$$

where n=number of bits

CHAPTER 4

DECODER OF IRMC ALGORITHM

A decoder for error detection and correction using an improved redundancy matrix coding algorithm is a system designed to encode messages that may contain errors [1]. The decoding operation performed at receiver side of space communications, which corrected all the errors using syndrome detection, error location detection, and error correction modules. The encoder uses the parity check matrix generated during the encoding process to detect and correct errors in the received message.

The encoding process begins with the received message, which is usually sent or stored as a binary string. The first step in the encoding process is to perform a similarity check on the received message using the similarity matrix created in the encoding process.

The equation matrix is a square matrix derived from the original message and the redundancy matrix. This is used to check the integrity of the message by comparing the received message with the encoded message. If an error is detected, an equation checking matrix can be used to determine the location and type of error.

If a bit error is detected, the decoder can correct the error by inverting the bit. If multiple errors are detected, the decoder may request that the message be retransmitted or use more advanced error correction techniques. The algorithm is designed in a way to detect and correct multiple errors in the message. It uses a larger parity check matrix that contains more information than standard error detection and correction methods. This allows for more reliable error correction and improved message integrity. Detecting and correcting errors in messages provides an efficient and effective way to ensure accurate and reliable transmission of information. The block diagram of decoder is shown in the figure 4.1.

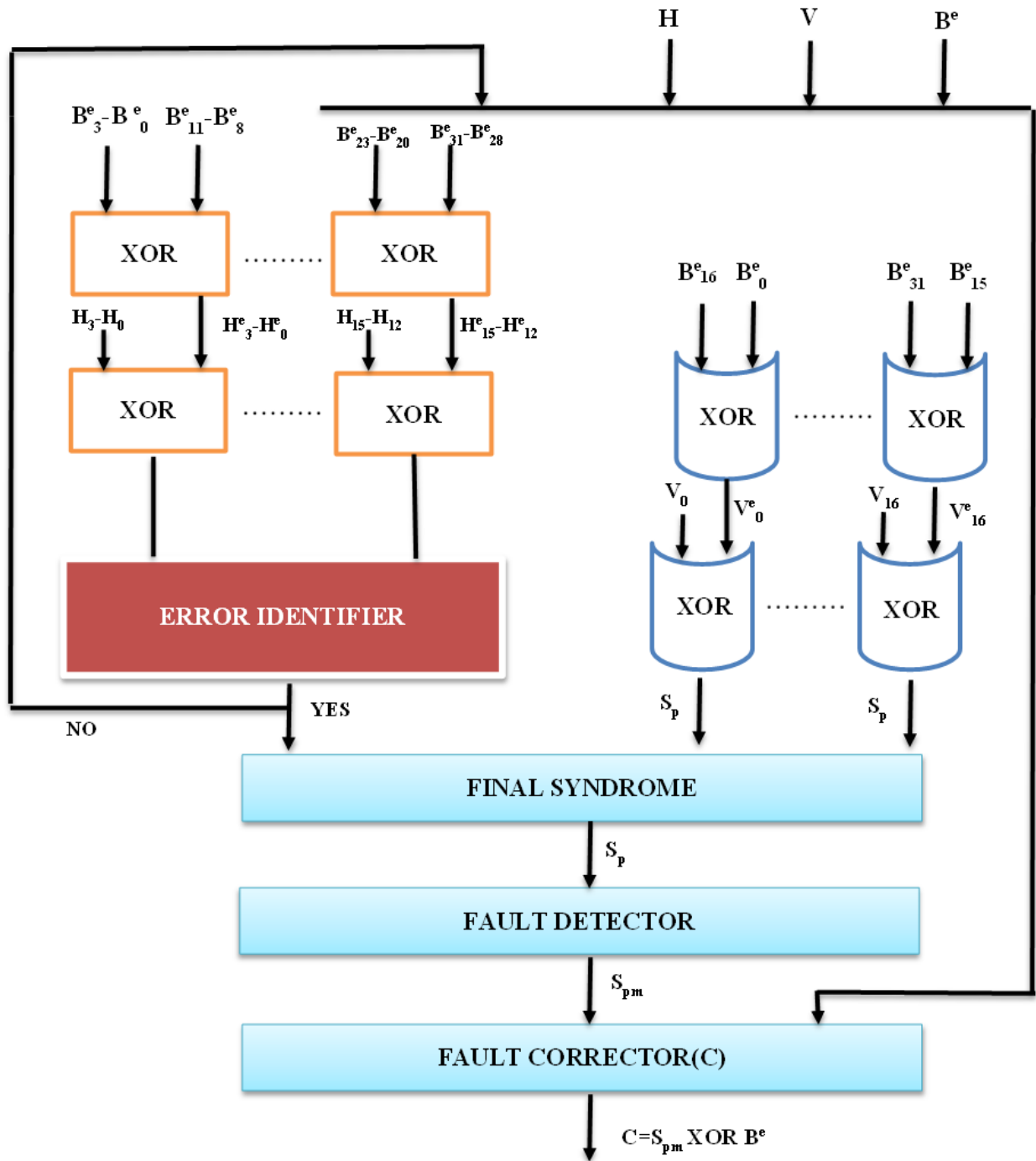


Figure 4.1: Block Diagram of IRMC Decoder

Error Detection and Correction using Improved Redundant Matrix Code

About the terms used in Decoder,

$B \rightarrow$ Original Input (consider 32-bit)

$H \rightarrow$ Horizontal redundant bits (16-bit)

$V \rightarrow$ Vertical redundant bits (16-bit)

$B^e \rightarrow$ Erroneous Input (32-bit)

$H^e \rightarrow$ Horizontal redundant bits for erroneous input (16-bit)

$V^e \rightarrow$ Vertical redundant bits for erroneous input (16-bit)

$S_p \rightarrow$ Syndrome value (16-bit)

$S_{pm} \rightarrow$ Updated syndrome value (32-bit)

$C \rightarrow$ Final modified output with no error bits (32-bit)

The equations used in decoder are,

i. Equation used for error identifier (YES/NO),

$\text{if } H \wedge H^e = 0 \quad (3); \text{ no error exists (NO)}$

$\text{if } H \wedge H^e \neq 0 \quad (4); \text{ error exists (YES)}$

ii. Equation used to generate syndrome,

$$S_p = V \wedge V^e \quad (5)$$

iii. Equation used to result the error less output code,

$$C = S_{pm} \wedge B^e \quad (6)$$

In Error detection and correction, the fault/error detector in the decoder plays an important role in detecting and correcting errors that may occur in data transmission. The enhanced redundancy matrix code algorithm is a special algorithm used to improve the error correction capability of such codes.

The algorithm works by adding additional bits to the original data bits, allowing the receiver to detect and correct errors that may occur during transmission. An error detector in the decoder then uses these additional redundant bits to determine which bits are damaged and uses appropriate correction techniques to recover the original data.

Syndrome refers to a set of values calculated based on the data received by the decoder and the extra bits added during encoding. This syndrome is then used to determine if errors occur during the transition, and if not, where they are located.

Error Detection and Correction using Improved Redundant Matrix Code

To calculate the syndrome, the decoder performs mathematical operations on the received data with redundant bits, usually by multiplying by a predetermined matrix. The result of this process is a set of values that can be compared with a set of expected (predefined) values to determine whether an error has occurred.

If the syndrome indicates that there is an error, the decoder can use several methods to correct the error. For example, if the syndrome indicates that a particular bit is in error, the decoder can move the bit to correct the error. Alternatively, the decoder may use more sophisticated techniques such as iterative decoding or maximum likelihood decoding to correct multiple errors.

In general, the syndrome is an important part of the error detection and correction process in the code using the improved coding redundancy matrix algorithm. By calculating and analyzing syndromes, the decoder can identify and correct errors that may occur during transmission, ensuring that the data received is accurate and reliable.

CHAPTER 5

WORKING OF IRMC ALGORITHM

5.1 FLOW OF PROCESS TO IDENTIFY AND RECTIFY ERROR BITS FOR 32-BIT ORIGINAL BIT FOR ENCODING AND 32-BIT ERRONEOUS BIT FOR DECODING

Consider Original input B,

B (31 st - 0 th)	1111	0101	1010	1111	1111	0110	1010	1100
-	S ₇	S ₆	S ₅	S ₄	S ₃	S ₂	S ₁	S ₀

Generating horizontal redundancy parameters $H(15^{\text{th}} - 0^{\text{th}})$ for original input B of 32-bit,

$$H_3 - H_0 = S_2 \wedge S_0 = 0110 \wedge 1100 = 1010$$

$$H_7 - H_4 = S_3 \wedge S_1 = 1111 \wedge 1010 = 0101$$

$$H_{11} - H_8 = S_6 \wedge S_4 = 0101 \wedge 1111 = 1010$$

$$H_{15} - H_{12} = S_7 \wedge S_5 = 1111 \wedge 1010 = 0101$$

$$\text{Therefore, } H(15^{\text{th}} - 0^{\text{th}}) = 0101 \ 1010 \ 0101 \ 1010$$

Generating vertical redundancy parameters $V(15^{\text{th}} - 0^{\text{th}})$ for original input B of 32-bit,

$$V(15^{\text{th}} - 0^{\text{th}}) = B(31^{\text{st}} - 16^{\text{th}}) \wedge B(15^{\text{th}} - 0^{\text{th}}) = 1111010110101111 \wedge 1111011010101100$$

$$\text{Therefore, } V(15^{\text{th}} - 0^{\text{th}}) = 0000 \ 0011 \ 0000 \ 0011$$

Encoding is done after generation of vertical and horizontal redundant bits of original input code.

Consider, Erroneous input,

$$B^e = 1111 \ 1101 \ 1010 \ 1101 \ 1101 \ 0100 \ 0111 \ 1100$$

Generating horizontal & vertical redundant parameters for B^e

$$H^e = 0101 \ 0000 \ 1010 \ 1000$$

$$V^e = 0010 \ 1001 \ 1101 \ 0001$$

After obtaining H, H^e , V and V^e

$$H \wedge H^e = 0000 \ 1010 \ 1111 \ 0010 (\neq 0)$$

Since, error exists, we need to calculate Syndrome (S_p).

$$S_p = V \wedge V^e = 0010 \ 1010 \ 1101 \ 0010$$

$$S_{pm} = 0010 \ 1010 \ 1101 \ 0010 \ 0010 \ 1010 \ 1101 \ 0010$$

S_{pm} is simply generated by writing 16-bit S_p two times in a row.

Error Detection and Correction using Improved Redundant Matrix Code

Using S_{pm} , we rectify the error to obtain errorless output using $C = B^e \wedge S_{pm}$.

Hence, Corrected bit (C) = 1111 0101 1010 1111 1111 0110 1010 1100 (31st to 0th bit).

		31 st bit		15 th bit													0 th bit
B	{	1	1	1	1	0	1	1	0	1	0	1	0	1	1	0	0
		1	1	1	1	0	1	0	1	1	0	1	0	1	1	1	1
V	→	0	0	0	0	0	0	1	1	0	0	0	0	0	0	1	1
B'	{	1	1	0	1	0	1	0	0	0	1	1	1	1	1	0	0
		1	1	1	1	1	1	0	1	1	0	1	0	1	1	0	1
V'	→	0	0	1	0	1	0	0	1	1	1	0	1	0	0	0	1
S _p	→	0	0	1	0	1	0	1	0	1	1	0	1	0	0	1	0
C	{	1	1	1	1	0	1	1	0	1	0	1	0	1	1	0	0
		1	1	1	1	0	1	0	1	1	0	1	0	1	1	1	1

Figure 5.1: Error bit correction

Figure 5.1 shows how the error bits are getting corrected using 2D forms. The figure 5, also describes what are the errors located in the erroneous input in order to understand how the process is getting executed. Syndrome is S_p , is calculated by XORing ($\wedge$) V and V_e .

CHAPTER 6

RESULTS

6.1 IMPLEMENTATION OF IRMC ALGORITHM USING VERILOG HDL LANGUAGE FOR 32-BIT INPUT

Simulation results, Design report & summary and timing details of 32-bit encoded & decoded result are provided in the coming figures:

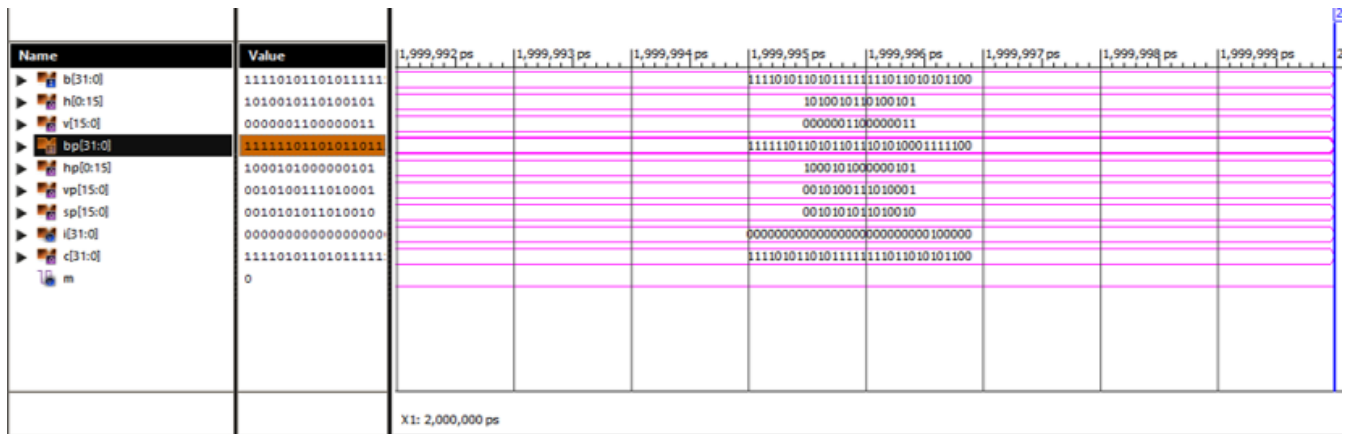


Figure 6.1.1: Simulation result of 32-bit input

majorproject_b17_2 Project Status (11/25/2022 - 16:14:24)			
Project File:	majorproject_b17_2.xise	Parser Errors:	No Errors
Module Name:	MajorProject_B17	Implementation State:	Synthesized
Target Device:	xc6slx4-3csg225	• Errors:	No Errors
Product Version:	ISE 14.2	• Warnings:	1 Warning (1 new)
Design Goal:	Balanced	• Routing Results:	
Design Strategy:	Xilinx Default (unlocked)	• Timing Constraints:	
Environment:	System Settings	• Final Timing Score:	

Device Utilization Summary (estimated values)				[-]
Logic Utilization	Used	Available	Utilization	
Number of Slice LUTs	112	2400		4%
Number of fully used LUT-FF pairs	0	112		0%
Number of bonded IOBs	176	132		133%

Figure 6.1.2: Design report details of 32-bit input

```

Partition Implementation Status
-----

No Partitions were found in this design.
-----

=====
*                               Design Summary                               *
=====

Top Level Output File Name      : MajorProject_B17.ngc

Primitive and Black Box Usage:
-----
# BELS                          : 112
#   LUT2                        : 64
#   LUT4                        : 48
# IO Buffers                    : 176
#   IBUF                       : 64
#   OBUF                       : 112

Device utilization summary:
-----

Selected Device : 6slx4csg225-3

```

Figure 6.1.3: Design summary of 32-bit input

```

Timing Details:
-----

All values displayed in nanoseconds (ns)

=====

Timing constraint: Default path analysis
Total number of paths / destination ports: 320 / 112
-----

Delay:          5.537ns (Levels of Logic = 3)
Source:         b<15> (PAD)
Destination:    sp<15> (PAD)

Data Path: b<15> to sp<15>



| Cell:in->out | fanout | Gate Delay                                                           | Net Delay | Logical Name (Net Name)  |
|--------------|--------|----------------------------------------------------------------------|-----------|--------------------------|
| IBUF:I->O    | 5      | 1.222                                                                | 0.962     | b_15_IBUF (b_15_IBUF)    |
| LUT4:I0->O   | 1      | 0.203                                                                | 0.579     | Mmux_c<31>11 (c_31_OBUF) |
| OBUF:I->O    |        | 2.571                                                                |           | c_31_OBUF (c<31>)        |
| Total        |        | 5.537ns (3.996ns logic, 1.541ns route)<br>(72.2% logic, 27.8% route) |           |                          |


```

Figure 6.1.4: Timing details of 32-bit input

Figure 6.1.1 shows whether the erroneous 32-bit is getting equal to original 32-bit after getting decoded. Figure 6.1.2 gives the synthesis and utilization design report of the project which is useful in analysing power. Whereas, figures 6.1.3 & 6.1.4 reports the design summary and timing details of the design, which is used to observe delay & latency of the design.

6.2 IMPLEMENTATION OF IRMC ALGORITHM USING VERILOG HDL LANGUAGE FOR 64-BIT INPUT

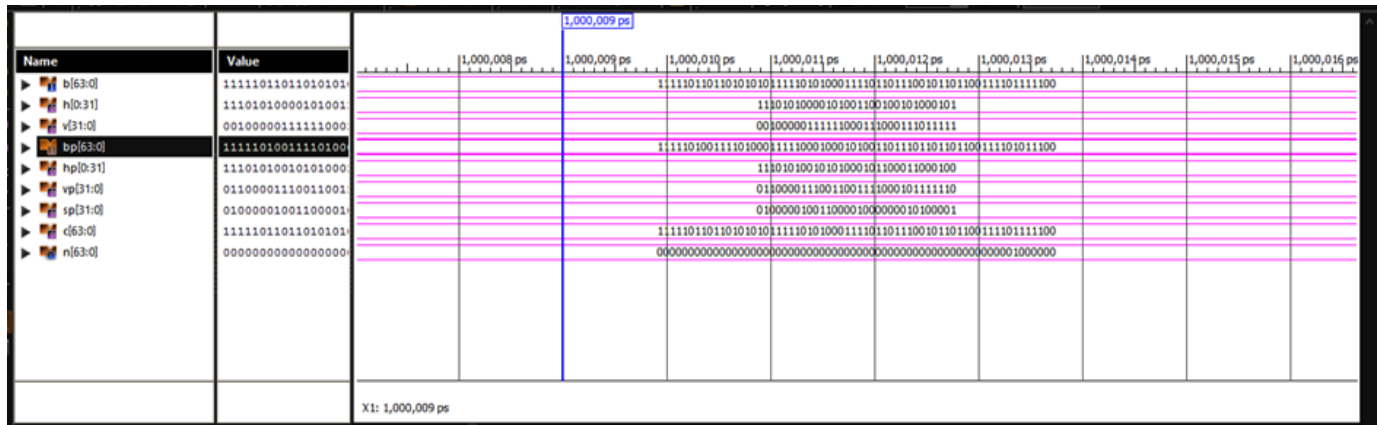


Figure 6.2.1: Simulation result of 64-bit input

MajorProject_B17 Project Status (11/25/2022 - 16:01:19)			
Project File:	MajorProject_B17.xise	Parser Errors:	No Errors
Module Name:	MajorProject_B17	Implementation State:	Synthesized
Target Device:	xc6slx4-3csg225	• Errors:	No Errors
Product Version:	ISE 14.2	• Warnings:	1 Warning (1 new)
Design Goal:	Balanced	• Routing Results:	
Design Strategy:	Xilinx Default (unlocked)	• Timing Constraints:	
Environment:	System Settings	• Final Timing Score:	

Device Utilization Summary (estimated values)			
Logic Utilization	Used	Available	Utilization
Number of Slice LUTs	224	2400	9%
Number of fully used LUT-FF pairs	0	224	0%
Number of bonded IOBs	352	132	266%

Detailed Reports					
Report Name	Status	Generated	Errors	Warnings	Infos
Synthesis Report	Current	Fri 25. Nov 16:01:17 2022	0	1 Warning (1 new)	0
Translation Report					
Map Report					
Place and Route Report					

Figure 6.2.2: Design report details of 64-bit input

Error Detection and Correction using Improved Redundant Matrix Code

```
Partition Implementation Status
-----

No Partitions were found in this design.
-----

=====
*                               Design Summary                               *
=====

Top Level Output File Name      : MajorProject_B17.ngc

Primitive and Black Box Usage:
-----
# BELS                          : 224
#   LUT2                        : 128
#   LUT4                        : 96
# IO Buffers                    : 352
#   IBUF                       : 128
#   OBUF                       : 224

Device utilization summary:
-----

Selected Device : 6slx4csg225-3
```

Figure 6.2.3: Design Summary of 64-bit

```
Maximum output required time after clock: No path found
Maximum combinational path delay: 5.537ns

Timing Details:
-----
All values displayed in nanoseconds (ns)

=====
Timing constraint: Default path analysis
Total number of paths / destination ports: 640 / 224
-----

Delay:          5.537ns (Levels of Logic = 3)
Source:         b<31> (PAD)
Destination:    sp<31> (PAD)

Data Path: b<31> to sp<31>

      Cell:in->out      fanout      Gate      Net
      Delay            Delay        Delay      Delay      Logical Name (Net Name)
-----
      IBUF:I->O          5      1.222      0.962      b_31_IBUF (b_31_IBUF)
      LUT4:I0->O          1      0.203      0.579      Mmux_c<63>11 (c_63_OBUF)
      OBUF:I->O          2.571      2.571      0.579      c_63_OBUF (c<63>)
-----
Total                    5.537ns (3.996ns logic, 1.541ns route)
                        (72.2% logic, 27.8% route)
```

Figure 6.2.4: Timing details of 64-bit input

Figure 6.2.1 shows whether the erroneous 64-bit is getting equal to original 64-bit after getting decoded. Figure 6.2.2 gives the synthesis and utilization design report of the project which is useful in analysing power. Whereas, figure 6.2.3 & 6.2.4 reports the design summary and timing details of the design, which is used to observe delay & latency of the design.

6.3 COMPARISON RESULTS OF 32-BIT INPUT

TABLE 6.3.1: COMPARISON OF DELAY AND POWER AMONG EXISTING ALGORITHMS & PROPOSED ALGORITHM (IRMC) FOR 32-BIT INPUT

Methods	Delay (ns)	Power (mW)
Reed-Muller Code ^[1]	55ns	264.8mW
Bit-Manipulation Algorithm ^[10]	6.065ns	327.3mW
Decimal Matrix Code ^[9]	17.587ns	121mW
IRMC (proposed)	5.937ns	38.903mW

Graphical representation of 32-bit comparisons is described in figures 6.3.1 and 6.3.2:

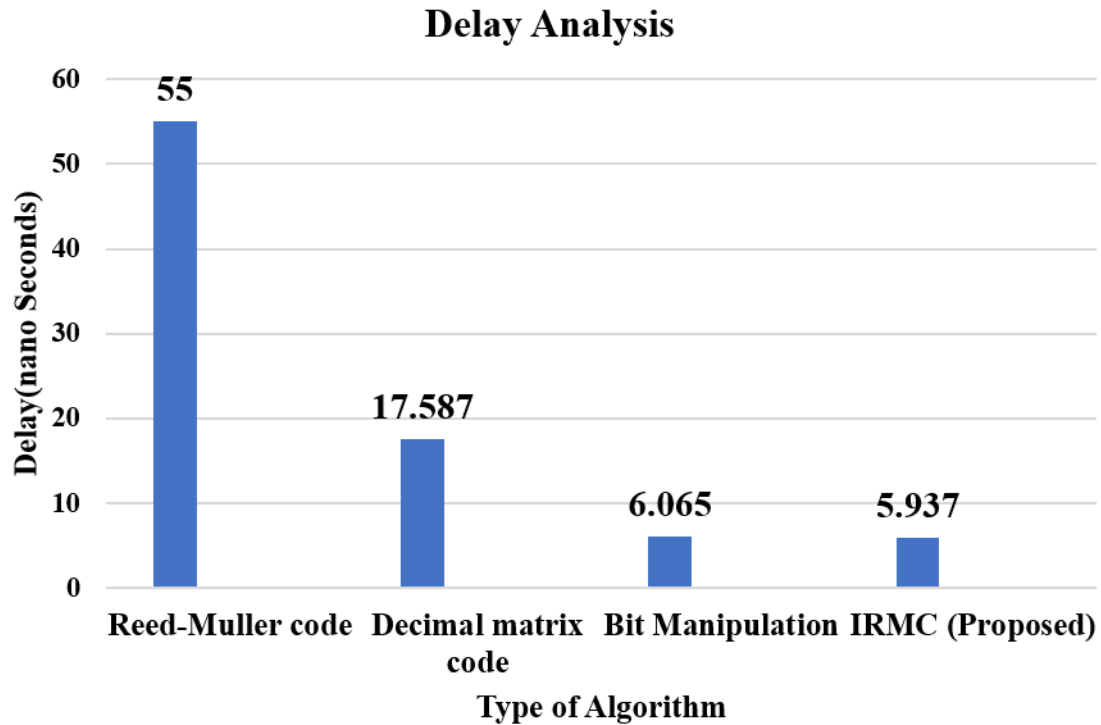


Figure 6.3.1: Graphical representation of delay analysis for 32-bit input

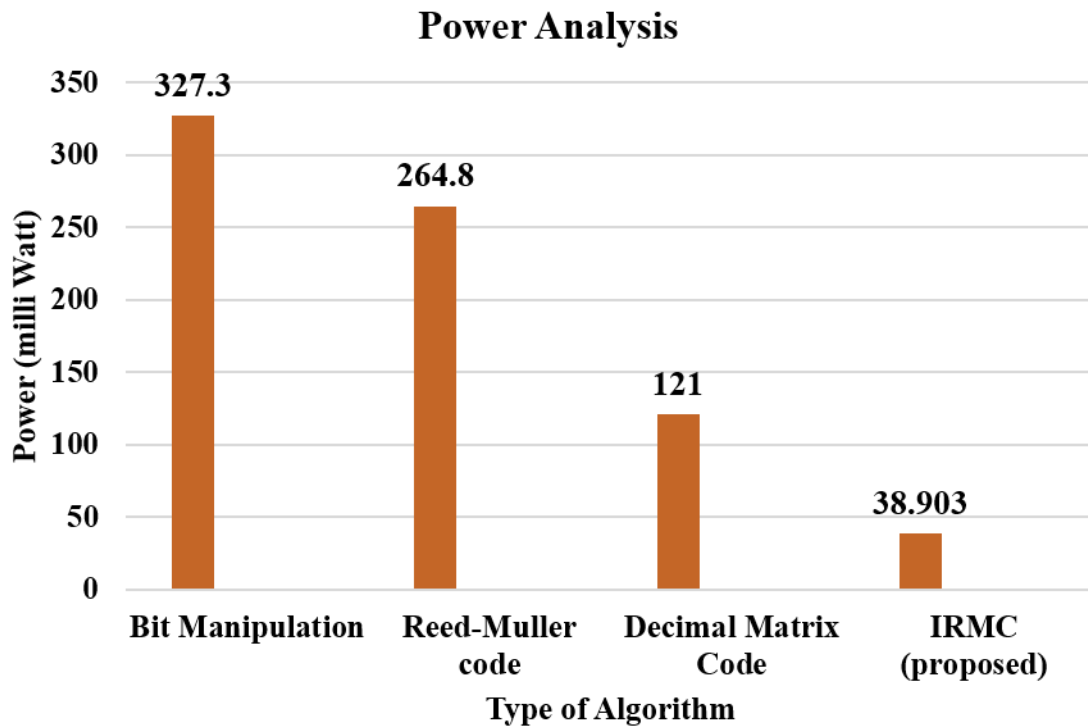


Figure 6.3.2: Graphical representation of power analysis for 32-bit input

6.4 COMPARISON RESULTS OF 64-BIT INPUT

TABLE 6.4.1: COMPARISON OF DELAY AND POWER AMONG EXISTING ALGORITHMS & PROPOSED ALGORITHM (IRMC) FOR 64-BIT INPUT

Methods	Power (mW)	Delay (nano seconds)
Decimal Matrix Code Algorithm ^[10]	330.8mW	8.492ns
Bit Manipulation Algorithm ^[10]	330.7mW	5.699ns
IRMC (proposed)	68.7mW	5.537ns

For 64-bit input, the proposed IRMC produced 5.537ns as delay and 68.7mW as power. Graphical representation of 64-bit comparisons are described in figure 6.4.1 and figure 6.4.2:

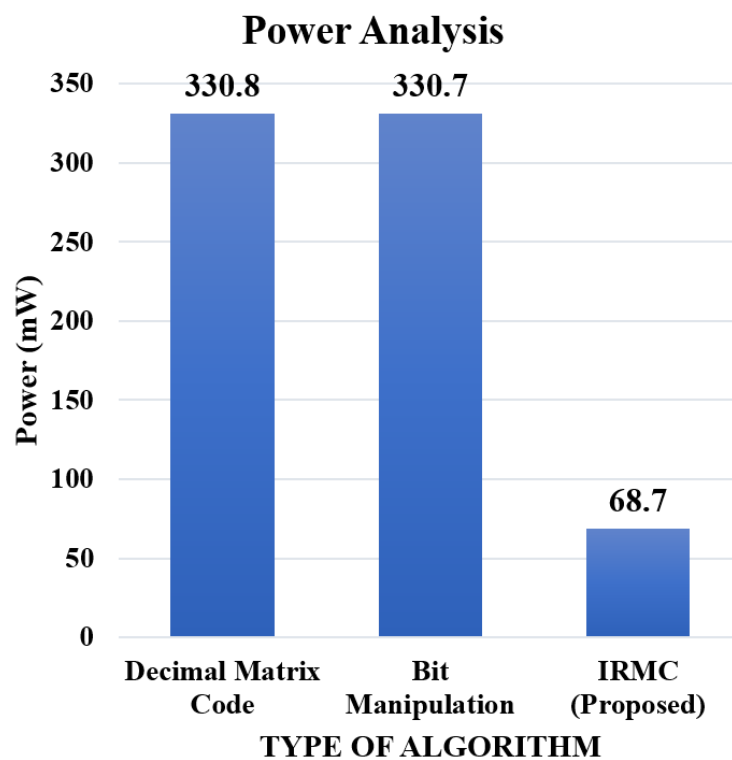


Figure 6.4.1: Graphical representation of power analysis for 64-bit input

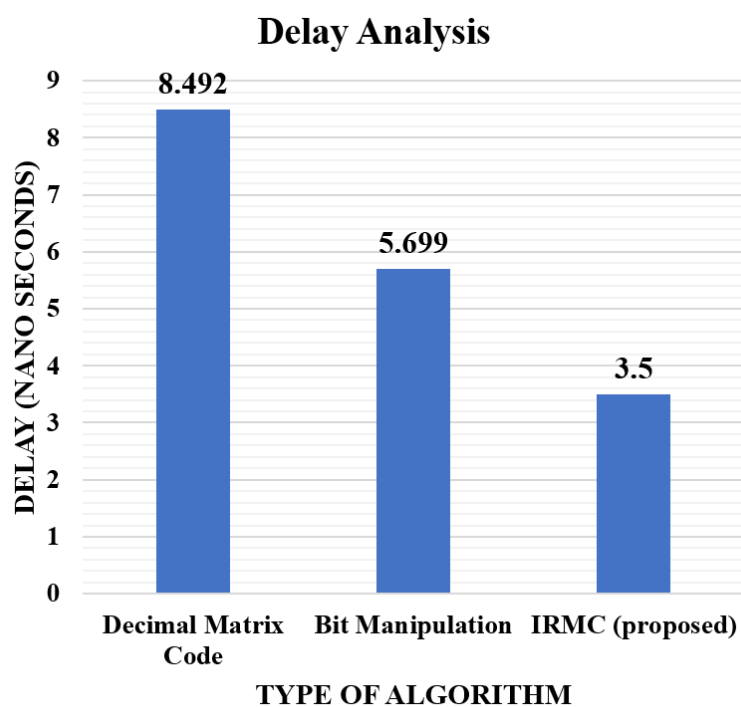


Figure 6.4.2: Graphical representation of delay analysis for 64-bit input

CHAPTER 7

CONCLUSION AND FUTURE SCOPE

7.1 CONCLUSION

The Improved Redundant Matrix Code algorithm has proven to be an effective method for error detection in both 32-bit and 64-bit inputs. By adding redundancy to the original data and performing parity checks, the algorithm can accurately detect and correct errors in the input data. The use of this algorithm can greatly improve the reliability of data transmission and storage systems. With the increasing amount of data being processed and stored, it is essential to have effective error detection mechanisms in place to ensure data integrity and prevent data loss. The Improved Redundant Matrix Code algorithm can also be easily implemented in hardware and software systems, making it a versatile solution for a wide range of applications. Its simplicity and efficiency make it a practical choice for many industries, including telecommunications, aerospace, and healthcare.

7.2 FUTURE SCOPE

An Improved Redundancy Matrix code algorithm provides a reliable and efficient approach to error detection and correction in VLSI. With the increasing demand for reliable digital communication systems, these algorithms are expected to play an important role in improving the performance and reliability of future systems such as integration with emerging technologies such as 5G, optimization of low-power devices, and implementation in various hardware electronic systems etc.

REFERENCES

- [1] M. Priya and M. M. Vijay, "Error Detection and Correction for SRAM Systems Using Improved Redundant Matrix Code," 2019 International Conference on Recent Advances in Energy-efficient Computing and Communication (ICRAECC), Nagercoil, India, 2019, pp. 1-8, doi: 10.1109/ICRAECC43874.2019.8995121.
- [2] R. Jenila, C. Supraja, C. K. Pappa and N. Dharani, "VLSI Implementation of Error Detection and Correction Codes for Space Engineering," 2021 Third International Conference on Intelligent Communication Technologies and Virtual Mobile Networks (ICICV), Tirunelveli, India, 2021, pp. 569-573, doi: 10.1109/ICICV50876.2021.9388477.
- [3] B. Liu, "Error-detecting/correcting-code-based self-checked/corrected/timed circuits," 2010 NASA/ESA Conference on Adaptive Hardware and Systems, Anaheim, CA, USA, 2010, pp. 66-72, doi: 10.1109/AHS.2010.5546217.
- [5] Helia Naeimi and André DeHon (2009) 'Fault Secure Encoder and Decoder for Nano Memory Applications', IEEE Transactions on Very Large-Scale Integration (VLSI) Systems, Vol.17, No.4, pp. 473-486.
- [4] K. Kaviya Priya, Mr. M. Palaniappan, "Implementation of Error Detection and Correction Codes using VLSI", SSRG International Journal of Electronics and Communication Engineering (SSRG – IJECE) – Special Issue ICRTCRET Mar 2019.
- [5] Zhang, X.. (2017). VLSI Architectures for Modern Error-Correcting Codes. 10.1201/b18673.
- [6] P. Schulz, A. Traßl, A. N. Barreto and G. Fettweis, "Efficient and Reliable Wireless Communications via Multi-Connectivity Using Rateless Codes in Single- and Multi-User Scenarios," in IEEE Transactions on Wireless Communications, vol. 20, no. 9, pp. 5714-5729, Sept. 2021, doi: 10.1109/TWC.2021.3069669.
- [7] F. M. Hashemian, M. Shivaie, and A. Daneshtalab, "A Low-Complexity and High-Performance Error Correction Code for Non-Volatile Memories," IEEE Transactions on Very Large Scale Integration (VLSI) Systems, vol. 27, no. 10, pp. 2225-2236, Oct. 2019.
- [8] Y. Chen, H. Zhang, Y. Song, and D. Chen, "Low-Latency Polar Codes for HARQ Feedback in 5G Systems," IEEE Transactions on Vehicular Technology, vol. 68, no. 6, pp. 5641-5645, Jun. 2019.
- [9] E. Abinaya, Dr. D. Somasundareswari, S. Mathan Prasad (2015) 'Error Detection and Correction in 32-bit SRAM based FPGA memory using Decimal Matrix Code', International Journal of Science Technology & Management, Vol.04, No.1, pp. 2394-1529.
- [10] V. Bhanumathi, M. Sunandini (2019) 'VLSI Architecture for error detection and correction based on XOR against multiple cell upsets with reduced redundant bits', ICTACT Journal of Microelectronics, Vol.05, No.2, pp. 2395-1680.
- [11] V. Yatskiv, S. Kulyna, P. Bykovyy, T. Maksymyuk and A. Sachenko, "Method of Reliable Data Storage Based on Redundant Residue Number System," 2020 IEEE 5th International Symposium on Smart and Wireless Systems within the Conferences on Intelligent Data Acquisition and Advanced Computing

Systems (IDAACS-SWS), Dortmund, Germany, 2020, pp. 1-4, doi: 10.1109/IDAACS-SWS50031.2020.9297052.

[12] T. P. Le, P. H. Trong, and T. Q. Dang, "High-Throughput Syndrome-Based Error Correction Codes for NAND Flash Memory," IEEE Transactions on Very Large Scale Integration (VLSI) Systems, vol. 28, no. 9, pp. 2116-2120, Sep. 2020.

[13] H. Heidarzadeh, S. E. Salami, and H. F. Rashvand, "Optimized Polar Codes for Beyond-5G Terahertz Communication Systems," IEEE Transactions on Communications, vol. 68, no. 2, pp. 1186-1195, Feb. 2020.

[14] T. Wang, H. Liu, J. Lin, and C. Wang, "An Efficient and Low-Complexity Error Correction Scheme for Non-Volatile Memory," IEEE Transactions on Very Large Scale Integration (VLSI) Systems, vol. 28, no. 5, pp. 1139-1143, May 2020.

[15] J. Zhou, Z. Liu, and Y. Li, "An Efficient Algorithm for Low-Density Parity-Check Codes Based on Min-Sum Message-Passing," IEEE Transactions on Communications, vol. 67, no. 6, pp. 4224-4234, Jun. 2019.

[16] M. Xiao, M. Liu, H. Li, and Y. Li, "Polar Codes for Channel Coding in DNA Storage Systems," IEEE Transactions on Nanobioscience, vol. 19, no. 2, pp. 179-186, Jun. 2020.

APPENDIX-A

PUBLICATION STATUS

- ❖ G. Shanthi, Aruru Sai Kumar, Uppala Bhargava Sai, Shaik Sajida, Gutta Karthikeya authored a paper titled, “Tunable design of OTA² – C² Low Pass Filter using CMOS Technology: An Innovative Approach” in 2023 4th International Conference for Emerging Technology (INCET) to be held at Belgaum, Karnataka on 26th – 28th May, 2023. Paper got accepted and registered. Paper to be presented on 28th May 2023. (**Under presentation**)
- ❖ G. Shanthi, Aruru Sai Kumar, Uppala Bhargava Sai, Shaik Sajida, Gutta Karthikeya authored a paper titled, “Error Detection and Correction code using Improved Redundant Matrix Code (IRMC) Algorithm” is drafted and to be submitted for journal. (**Drafted**)

Error Detection and Correction code using Improved Redundant Matrix Code (IRMC) Algorithm

G. Shanthi¹ . A. Sai Kumar² . U. Bhargava Sai³ . Sk. Sajida⁴ . G. Karthikeya⁵

Abstract

Mobile way communication over the Internet requires error detection and correction. Error Control describes how networks handle and detect errors at the data link layer. Each network connection is assumed to have two channels. One for data traffic and one for signaling and control. Robust error detection is encoded on the traffic channel based on the information bits and the correction code to form the transmitted data stream. Currently, the error is tolerated by the SRAM memory frame, so Single Cell Upsets (SCU) or Multiple Cell Upsets (MCU) can occur. One of the main reasons for the MCU is the astronomical radiation. The proposed technique is called Improved Redundant Matrix Code to protect SRAM-based memories from the detuning of a large number of components. The Improved Redundancy Matrix Code (IRMC) is used to detect and troubleshoot storage devices. IRMC's detection rate and adjustments are in contrast to other rescue codes. Space, power, and defer overhead is only. Results should prove that IRMC has higher quality and higher correctability for MCU tuned for the lower power and delay. Xilinx 14.7 is the tool used for the process and using Verilog HDL, programming of the algorithm is performed.

Keywords: Error Correction Code (ECC), Multiple Cell Upsets (MCU), Improved Redundant Matrix Code (IRMC), Multiple error detection, Multiple error correction Memory.

1. INTRODUCTION

Systems for transportation and communication are two that provide services to consumers. Clients can quickly assemble data for these services through parallel transmission. During transmission, all data bits are sent simultaneously through different communication connections [1]. Here, a direct link is established between the number of active lines and the number of bits to send. Each group of bits is sent over the data lines on each clock pulse. Practically speaking, parallel transmission requires "N" communication channels to work. Contrary to popular belief, sequential transmission requires only one communication channel, which reduces the total cost compared to serial transmission [2]. Memories will be solid only if the information stored inside is error-free. If there is an error, you will know at once.

Communication difficulties that arise during strategy implementation are caused by transmission errors. Soft errors are those caused by external noise or radiation rather than design or manufacturing defects [3]. To communicate on the phone using the web, error detection and correction codes are required. Packages are received by the device through a port in the framework programs, processed with it,

and then transmitted to one or more output ports. Data in memory is checked while this handling is in progress [4]. To detect and correct errors, the stored data needs to be examined. The basic idea is to incorporate revision to correct errors and add information to the message that receivers can use to evaluate the relevance of the message.

When using a non-random code, the encoded message is modified to have a certain number of bits that are less likely. Another grouping is a type of systematic module, in which the transmitter sends unique data with a predetermined number of parity bits, such as check bits derived from information bits [5]. When only error detection is required, the beneficiary uses the same computation to compare the received information bits with the received check bits. If the attributes do not match, we infer that an error occurred at some point during the transmission process. Typically, error-correcting codes are used for lower-layer communication, equivalent to reliable limits in media such as CDs, DVDs, hard disks, and RAM [6-7].

A disadvantage of many error-detection algorithms such as CRC is that they can only detect errors. Once errors in the data are detected at the

destination, they are impossible to correct, and a new transmission of the data is required to receive the message [8]. Error-detection and correction algorithms have significant weaknesses in managing designs with low power and delay characteristics and eliminating redundancy. It is also important to guarantee decoder hardware around memory blocks that have become susceptible to soft errors [9]. It is used to identify and characterize a distinct class of error-correcting codes whose redundancy makes the setup of fault-secure detectors (FSD) particularly critical. In a situation where the structural failure rate is determined by the encoder and decoder failure rates, it also accounts for the importance of ensuring the encoder and decoder hardware against transient errors [10-11].

It uses Parity Check Matrix and Matrix Generator to encode the data and encode it by matrix multiplication and addition [12]. Parity matrix codes are designed to improve error correction capabilities with extra bits. This fix contains minor performance improvements and fixes most memory errors.

Many real-time applications like, Internet calls require error detection and correction. The term "error control" refers to the management and detection of errors in the data link layer of the network [13]. This is a common setting for using error correction code (ECC) in memory devices. The encoding/decoding circuit of ECCs used for data storage must be efficient in terms of bandwidth, power and delay [14]. The algorithm used in this study is called improved redundancy matrix coding (IRMC). IRMC is used to detect and repair defects in memory devices [15-16]. The range, power, and delay of other correction codes are comparable to IRMC's detection and correction speed [1].

The Improved redundancy matrix code (IRMC) algorithm is a coding technique used in Large Scale Integration (VLSI) technology to improve the reliability of digital circuits. This is a type of error correction code (ECC) that allows you to detect and correct errors that may occur during data transfer or storage [2].

2. IMPROVED REDUNDANT MATRIX CODE ALGORITHM

In the IRMC algorithm, a redundant bit matrix is added to the original data to generate a new code word [1]. This redundant bit matrix is carefully

designed to ensure that any bit errors are detected and corrected, and multiple bit errors are detected.

The IRMC algorithm is particularly useful in VLSI technology where the circuit density is very high and the possibility of errors is high. The use of IRMC code algorithm in VLSI can improve circuit reliability, reduce the risk of data corruption and improve overall system performance [1].

In addition, the IRMC algorithm is also used in other fields such as telecommunications, data storage, and satellite communications. This is a widely accepted method of error correction and its use is expected to increase as technology advances.

In the proposed work, the IRMC is executed for error detection and correction of 32-bit and 64-bit input codes using Verilog in Xilinx tool in the below mentioned stream of steps:

- i. Taking the Original 32-bit/ 64-bit as input for Encoder of IRMC algorithm.
- ii. Generating Horizontal Redundant bits using AXL calculation and Vertical Redundant bits using Logical Organization of Original input in Encoder region.
- iii. Taking Erroneous 32-bit/64-bit as input for decoder part.
- iv. Performing step (ii) for the erroneous input to get respective horizontal and vertical redundant bits.
- v. Generating Syndrome value to detect and correct the error, if error exists.
- vi. Using the generated syndrome value, the error can be detected and corrected, that resulting into the erroneous input bit will be regenerated, which should be same as that of original input bit.
- vii. Generating Simulation results & Design report using Xilinx tool.
- viii. Delay and Power Analysis of IRMC Algorithm.
- ix. Comparing the resulted IRMC algorithm delay and power with different algorithms.

3. ENCODER OF IRMC ALGORITHM

Using the Enhanced Redundancy Matrix Coding Algorithm, the error detection and correction encoder code involves adding additional bits to the original message to detect and correct errors that may occur during transmission or storage. This process involves creating a redundancy matrix code that can be used to detect errors and locate them in messages [1].

The Enhanced Redundancy Matrix Coding algorithm is an error detection and correction technique that can detect and correct multiple errors in a message. It works by creating a parity check matrix that is used to detect errors in received messages. The same matrix is created by adding additional rows and columns to the original message matrix [1]. These additional rows and columns contain parity bits used to check the integrity of the message.

To encode a message using the enhanced redundancy matrix encoding algorithm, the following steps are usually performed:

i. The message is converted into binary code.

ii. Binary messages are divided into blocks of equal size.

iii. The redundancy matrix is created by adding additional rows and columns to the original message matrix. These additional rows and columns contain parity bits used to check the integrity of the message.

iv. The message matrix and the redundancy matrix are multiplied together to produce the encoded message.

v. Coded messages are sent or saved.

If an error occurs during transmission or storage, the recipient can use the parity check matrix to determine the location and type of error. If the error is a single error, it can be corrected using the parity bit. If some errors are detected, the message may need to be sent.

Overall, the error detection and correction code encoder using the improved or enhanced redundancy matrix code algorithm provides an efficient method for detecting and correcting errors in messages. It is widely used in various applications such as communication systems, data storage systems, and digital media [1].

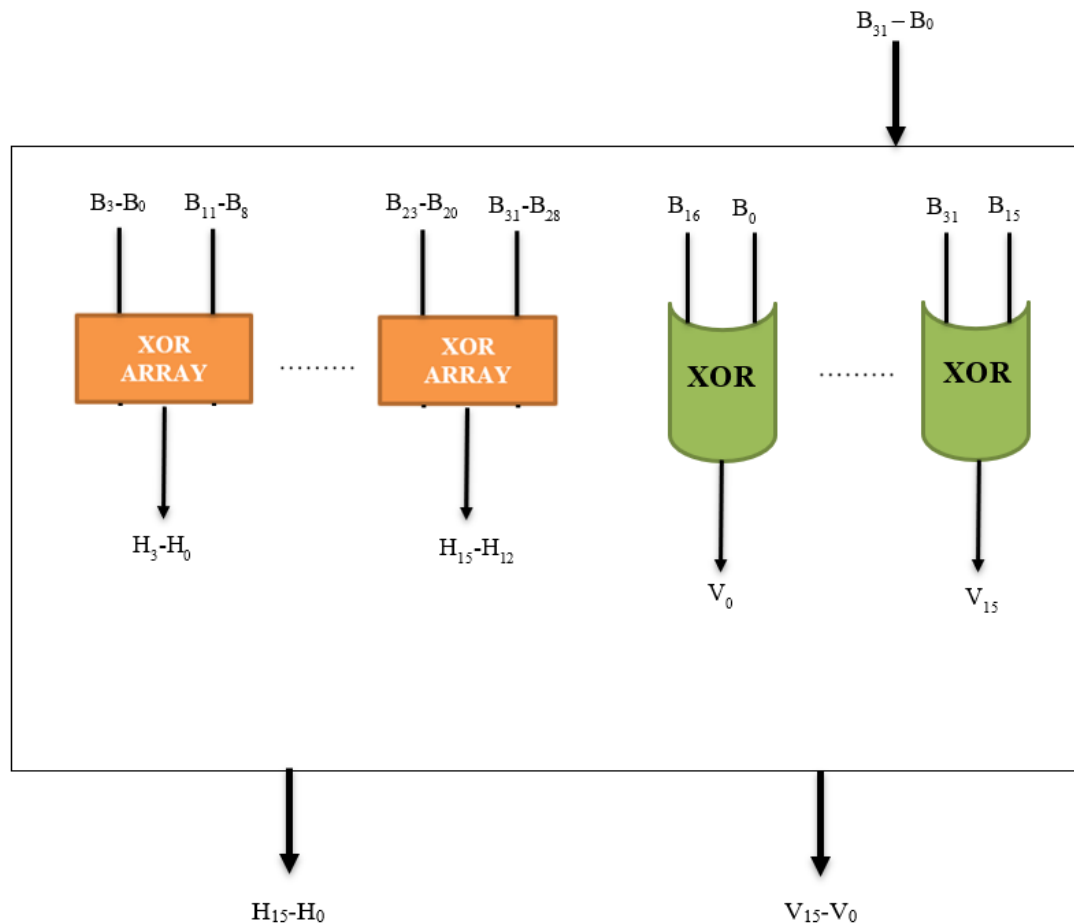


Fig. 1. Block Diagram of IRMC Encoder

Consider, an encoder was introduced that registers repeating bits using logic arrays and XOR inputs. Additional parts are added to distinguish or correct possible defects [1]. The figure 1 shows the horizontal redundant bits ($H_{15}-H_0$) and the vertical redundant bits ($V_{15}-V_0$) generated for the 32-bit input ($B_{31}-B_0$).

The algorithm in the IRMC encoder starts with the same matrix. Choosing the logic conditions to calculate each equation and syndrome is not difficult when the equation matrix is chosen. For example, the AXL calculation uses symbols S_0 ($B_3B_2B_1B_0$) and S_2 ($B_{11}B_{10}B_9B_8$) to obtain the extra bits [$H_3H_2H_1H_0$].

In the error detection and correction code encoder using the Enhanced Redundant Matrix Coding Algorithm, AXL refers to additional checks for functions such as XOR. A method used to calculate additional redundancy that can be used for error detection and correction.

The AXL computing process involves XORing different parts of the input data bits and then

combining the results to create additional redundant bits. Specifically, for a given data set, the encoder selects a set of indices that correspond to certain bits in the block, and then XORs the integer values together to produce a single checksum. This process is repeated for the majority of bits, and the resulting check values are summed and concatenated to produce the final AXL bit.

AXL bits are then added to the original data bits to create a new, larger data set that can be improved with error detection and correction capabilities. In the decoder, the AXL bit is used together with other redundant bits to calculate syntax, which is then used to detect and correct errors that may occur during transmission. In general, the AXL algorithm is an important component of the improved redundancy matrix code algorithm because it provides error detection and correction capabilities that can improve the accuracy and reliability of data transmission.

Figure 2 shows the Information redundancy and logical organization of original input (for 32-bit) and generated horizontal and vertical redundant bits:

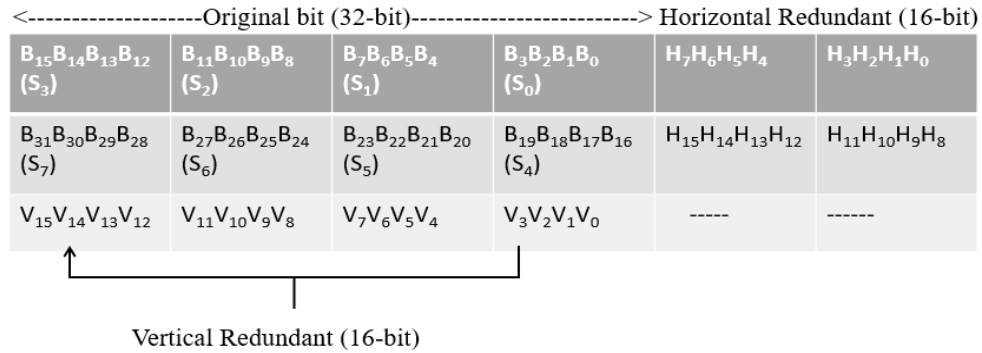


Fig. 2. Logical Organization of Original input and redundant bits

The horizontal and vertical redundant bits are computed in the below manner:

Horizontal Bits generation using AXL calculation,

$$B_{11}B_{10}B_9B_8 \wedge B_3B_2B_1B_0 = H_3H_2H_1H_0$$

$$B_{15}B_{14}B_{13}B_{12} \wedge B_7B_6B_5B_4 = H_7H_6H_5H_4$$

(1)

Vertical Bits generation,

$$V_n = B_n \wedge B_{n+16} \quad (2); \text{ where } n=\text{number of bits}$$

4. DECODER OF IRMC ALGORITHM

A decoder for error detection and correction using an improved redundancy matrix coding algorithm is a system designed to encode messages

that may contain errors [1]. The encoder uses the parity check matrix generated during the encoding process to detect and correct errors in the received message.

The encoding process begins with the received message, which is usually sent or stored as a binary string. The first step in the encoding process is to perform a similarity check on the received message using the similarity matrix created in the encoding process.

The equation matrix is a square matrix derived from the original message and the redundancy matrix. This is used to check the integrity of the message by comparing the received message with the

encoded message. If an error is detected, an equation checking matrix can be used to determine the location and type of error.

If a bit error is detected, the decoder can correct the error by inverting the bit. If multiple errors are detected, the decoder may request that the message be retransmitted or use more advanced error correction techniques. The algorithm is designed in a way to detect and correct multiple errors in the

message. The algorithm uses a larger parity check matrix that contains more information than standard error detection and correction methods. This allows for more reliable error correction and improved message integrity. Detecting and correcting errors in messages provides an efficient and effective way to ensure accurate and reliable transmission of information. The block diagram of decoder is shown below:

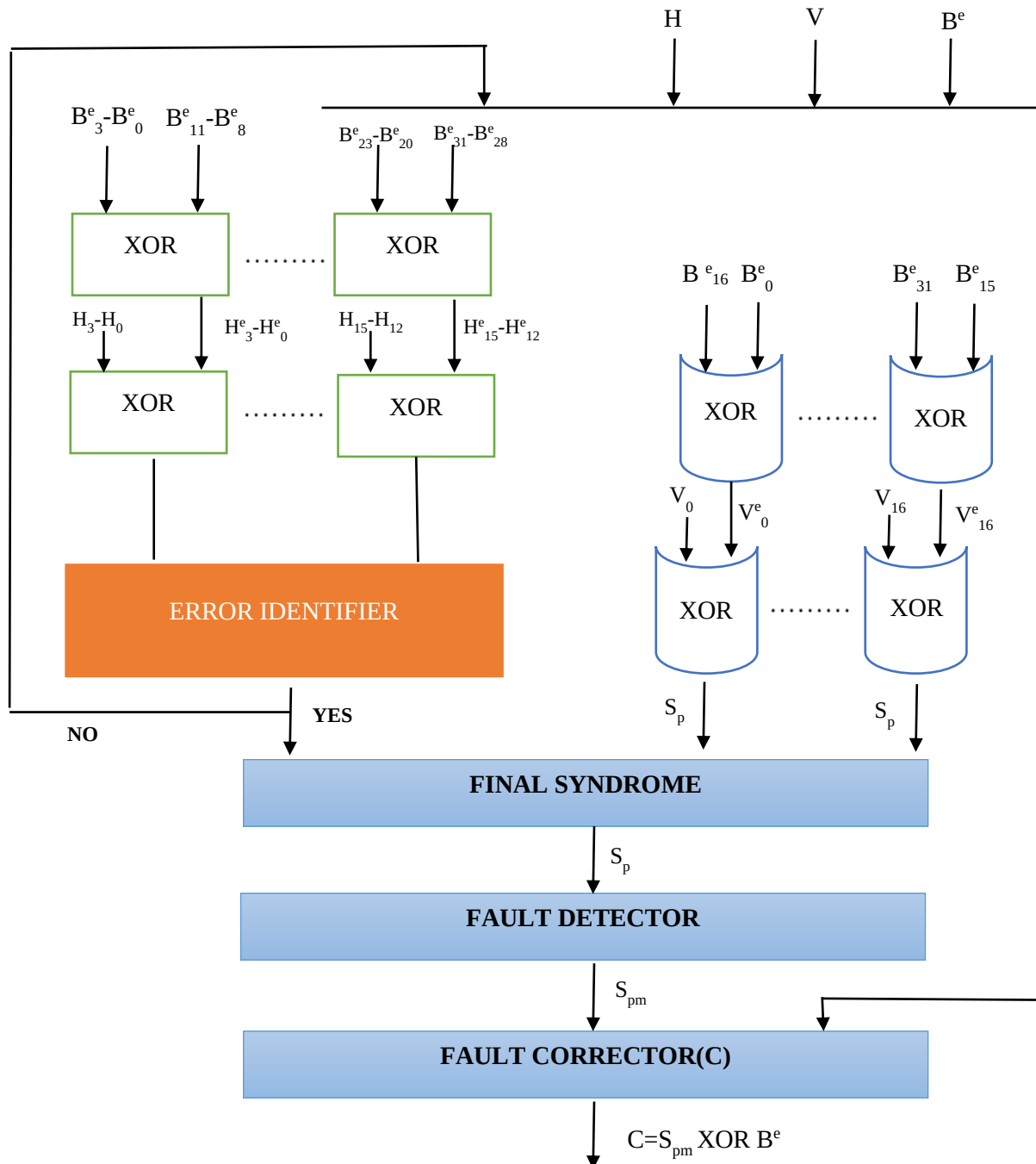


Fig. 3. Block Diagram of IRMC Decoder

About the terms used in Decoder,

$B \rightarrow$ Original Input (consider 32-bit)

$H \rightarrow$ Horizontal redundant bits (16-bit)

$V \rightarrow$ Vertical redundant bits (16-bit)

$B^e \rightarrow$ Erroneous Input (32-bit)

$H^e \rightarrow$ Horizontal redundant bits for erroneous input (16-bit)

$V^e \rightarrow$ Vertical redundant bits for erroneous input (16-bit)

$S_p \rightarrow$ Syndrome value (16-bit)

$S_{pm} \rightarrow$ Updated syndrome value (32-bit)

$C \rightarrow$ Final modified output with no error bits (32-bit)

The equations used in decoder are,

i. Equation used for error identifier (YES/NO),

if $H \wedge H^e = 0$ (3); no error exists (NO)

if $H \wedge H^e \neq 0$ (4); error exists (YES)

ii. Equation used to generate syndrome,

$$S_p = V \wedge V^e \quad (5)$$

iii. Equation used to result the error less output code,

$$C = S_{pm} \wedge B^e \quad (6)$$

Error detection and correction The error detector in the decoder plays an important role in detecting and correcting errors that may occur in data transmission. The enhanced redundancy matrix code algorithm is a special algorithm used to improve the error correction capability of such codes.

The algorithm works by adding additional bits to the original data bits, allowing the receiver to detect and correct errors that may occur during transmission. An error detector in the decoder then uses these additional redundant bits to determine which bits are damaged and uses appropriate correction techniques to recover the original data.

Syndrome refers to a set of values calculated based on the data received by the decoder and the extra bits added during encoding. This syndrome is then used to determine if errors occur during the transition, and if not, where they are located.

To calculate the syndrome, the decoder performs mathematical operations on the received data with redundant bits, usually by multiplying by a predetermined matrix. The result of this process is a set of values that can be compared with a set of expected (predefined) values to determine whether an error has occurred.

If the syndrome indicates that there is an error, the decoder can use several methods to correct the error. For example, if the syndrome indicates that a particular bit is in error, the decoder can move the bit to correct the error. Alternatively, the decoder may use more sophisticated techniques such as iterative decoding or maximum likelihood decoding to correct multiple errors.

In general, the syndrome is an important part of the error detection and correction process in the code using the improved coding redundancy matrix algorithm. By calculating and analyzing syndromes, the decoder can identify and correct errors that may occur during transmission, ensuring that the data received is accurate and reliable.

5. WORKING OF IRMC ALGORITHM FOR 32-BIT INPUT

Input Parameters & Flow of process to identify and rectify error bits,

Consider,

$B = 1111\ 0101\ 1010\ 1111\ 1111\ 0110\ 1010\ 1100$
(31st bit to 0th bit)

Generating horizontal & vertical parameters for original input B

$H = 0101\ 1010\ 0101\ 1010$

$V = 0000\ 0011\ 0000\ 0011$

Consider, Erroneous input,

$B^e = 1111\ 1101\ 1010\ 1101\ 1101\ 0100\ 0111\ 1100$

Generating horizontal & vertical parameters for B^e

$H^e = 0101\ 0000\ 1010\ 1000$

$V^e = 0010\ 1001\ 1101\ 0001$

After obtaining H, H^e , V and V^e

$H \wedge H^e = 0000\ 1010\ 1111\ 0010 (\neq 0)$

Since, error exists, we need to calculate Syndrome (S_p).

$$S_p = V \wedge V^e = 0010\ 1010\ 1101\ 0010$$

$$S_{pm} = 0010\ 1010\ 1101\ 0010\ 0010\ 1010\ 1101\ 0010$$

S_{pm} is simply generated by writing 16-bit S_p two times in a row.

Using S_{pm} , we rectify the error to obtain errorless output using $C = B^e \wedge S_{pm}$.

Hence, $C = 1111\ 0101\ 1010\ 1111\ 1111\ 0110\ 1010\ 1100$ (31st to 0th bit).

	31 st bit	1	1	1	1	0	1	1	0	1	0	1	0	1	1	0	0	0 th bit
B		1	1	1	1	0	1	0	1	1	0	1	0	1	1	1	1	16 th bit
V		0	0	0	0	0	0	1	1	0	0	0	0	0	0	1	1	
B ^e		1	1	0	1	0	1	0	0	1	1	1	1	1	1	0	0	
		1	1	1	1	1	1	0	1	1	0	1	0	1	1	0	1	
V ^e		0	0	1	0	1	0	0	1	1	1	0	1	0	0	0	1	
S _p		0	0	1	0	1	0	1	0	1	1	0	1	0	0	1	0	
C		1	1	1	1	0	1	1	0	1	0	1	0	1	1	0	0	
		1	1	1	1	0	1	0	1	1	0	1	0	1	1	1	1	

Fig. 5. Correction of error bits

Figure 5 shows how the error bits are getting corrected using 2D forms. The figure 5, also describes what are the errors located in the erroneous input in order to understand how the process is getting executed. Syndrome is S_p , is calculated by XORing($\wedge$) V and V^e .

6. RESULTS

6.1. Implementation of IRMC Algorithm using Verilog HDL language for 32-bit input

Simulation results, Design report & timing details of 32-bit encoded & decoded result are provided in the coming figures:

Name	Value	1,999,992 ps	1,999,993 ps	1,999,994 ps	1,999,995 ps	1,999,996 ps	1,999,997 ps	1,999,998 ps	1,999,999 ps
b[31:0]	11110101101011111				11110101101011111	11110101101011100			
h[0:15]	1010010110100101				1010010110100101				
v[15:0]	0000001100000011				0000001100000011				
bp[31:0]	1111101101011011				1111101101011011	101010001111100			
hp[0:15]	1000101000000101				1000101000000101				
vp[15:0]	0010100111010001				0010100111010001				
sp[15:0]	0010101011010010				0010101011010010				
i[31:0]	0000000000000000				0000000000000000	0000000000000000			
c[31:0]	11110101101011111				11110101101011111	11110101101011100			
m	0								

Fig. 6. Simulation result of 32-bit input

majorproject_b17_2 Project Status (11/25/2022 - 16:14:24)			
Project File:	majorproject_b17_2.xise	Parser Errors:	No Errors
Module Name:	MajorProject_B17	Implementation State:	Synthesized
Target Device:	xc6slx4-3csg225	Errors:	No Errors
Product Version:	ISE 14.2	Warnings:	1 Warning (1 new)
Design Goal:	Balanced	Routing Results:	
Design Strategy:	Xilinx Default (unlocked)	Timing Constraints:	
Environment:	System Settings	Final Timing Score:	

Device Utilization Summary (estimated values)			
Logic Utilization	Used	Available	Utilization
Number of Slice LUTs	112	2400	4%
Number of fully used LUT-FF pairs	0	112	0%
Number of bonded IOBs	176	132	133%

Fig.7. Design report details of 32-bit input

Timing Details:

All values displayed in nanoseconds (ns)

Timing constraint: Default path analysis
Total number of paths / destination ports: 320 / 112

Delay: 5.537ns (Levels of Logic = 3)
Source: b<15> (PAD)
Destination: sp<15> (PAD)

Data Path: b<15> to sp<15>

Cell:in->out	fanout	Gate Delay	Net Delay	Logical Name (Net Name)
IBUF:I->O	5	1.222	0.962	b_15_IBUF (b_15_IBUF)
LUT4:I0->O	1	0.203	0.579	Mmux_c<31>11 (c_31_OBUF)
OBUF:I->O		2.571		c_31_OBUF (c<31>)
Total		5.537ns		(3.996ns logic, 1.541ns route) (72.2% logic, 27.8% route)

Fig. 8. Timing details of 32-bit input

Figure 6 shows whether the erroneous 32-bit is getting equal to original 32-bit after getting decoded. Figure 7 gives the synthesis and utilization design report of the project which is useful in analysing power. Whereas Figure 8 reports the timing details of the design, which is used to observe delay & latency of the design.

6.2. Implementation of IRMC Algorithm for 64-bit input

Simulation results, Design report & timing details of 32-bit encoded & decoded result are provided in the coming figures:

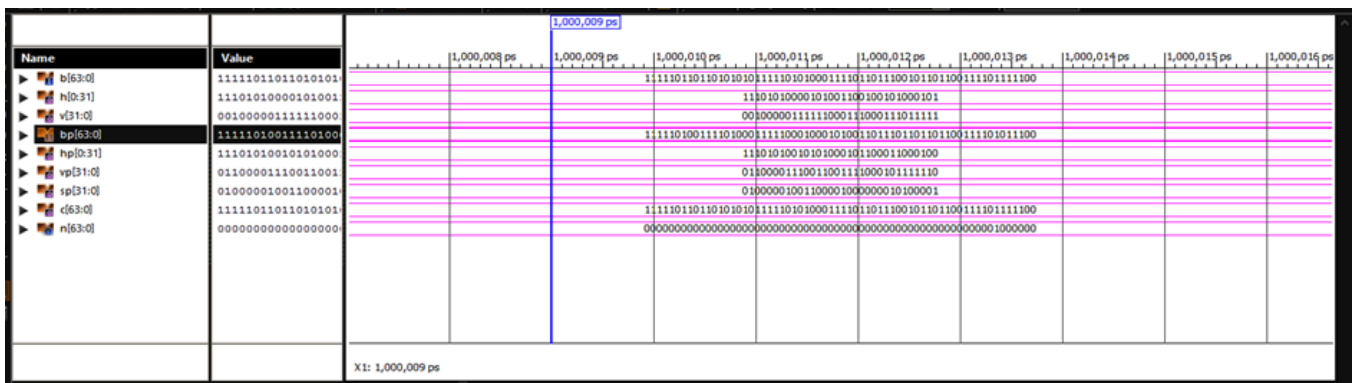


Fig. 9. Simulation result of 64-bit input

MajorProject_B17 Project Status (11/25/2022 - 16:01:19)			
Project File:	MajorProject_B17.xise	Parser Errors:	No Errors
Module Name:	MajorProject_B17	Implementation State:	Synthesized
Target Device:	xc6slx4-3csg225	Errors:	No Errors
Product Version:	ISE 14.2	Warnings:	1 Warning (1 new)
Design Goal:	Balanced	Routing Results:	
Design Strategy:	Xilinx Default (unlocked)	Timing Constraints:	
Environment:	System Settings	Final Timing Score:	

Device Utilization Summary (estimated values)			
Logic Utilization	Used	Available	Utilization
Number of Slice LUTs	224	2400	9%
Number of fully used LUT-FF pairs	0	224	0%
Number of bonded IOBs	352	132	266%

Fig. 10. Design report of 64-bit input

Maximum output required time after clock: No path found
Maximum combinational path delay: 5.537ns

Timing Details:

All values displayed in nanoseconds (ns)

Timing constraint: Default path analysis
Total number of paths / destination ports: 640 / 224

Delay: 5.537ns (Levels of Logic = 3)
Source: b<31> (PAD)
Destination: sp<31> (PAD)

Data Path: b<31> to sp<31>

Cell:in->out	fanout	Gate Delay	Net Delay	Logical Name (Net Name)
IBUF:I->O	5	1.222	0.962	b_31_IBUF (b_31_IBUF)
LUT4:I0->O	1	0.203	0.579	Mmux_c<63>11 (c_63_OBUF)
OBUF:I->O		2.571		c_63_OBUF (c<63>)
Total		5.537ns		(3.996ns logic, 1.541ns route) (72.2% logic, 27.8% route)

Fig. 11. Timing Details of 64-bit input

Figure 9 shows whether the erroneous 64-bit is getting equal to original 64-bit after getting decoded. Figure 10 gives the synthesis and utilization design report of the project which is useful in analysing power. Whereas Figure 11 reports the timing details of the design, which is used to observe delay & latency of the design.

7. COMPARISON OF IRMC WITH DIFFERENT EXISTING ALGORITHMS

For 32-bit input, the obtained delay value is 5.937ns and power obtained is 38.903mW.

Here, the obtained results of power & delay are compared with existing algorithms like DMC, BMC, OLS, etc

Comparison table of delay & power analysis for 32-bit input obtained using IRMC with other algorithms:

Table 1 Delay and Power comparison between IRMC (proposed) and other existing algorithms

Methods	Delay (ns)	Power (mW)
IRMC (proposed)	5.937ns	38.903mW
Decimal Matrix Code^[9]	17.587ns	121mW
Bit-Manipulation Algorithm^[10]	6.065ns	327.3mW
Reed-Muller Code^[1]	55ns	264.8mW

Graphical representation of 32-bit comparisons is described in figure 12 and figure 13:

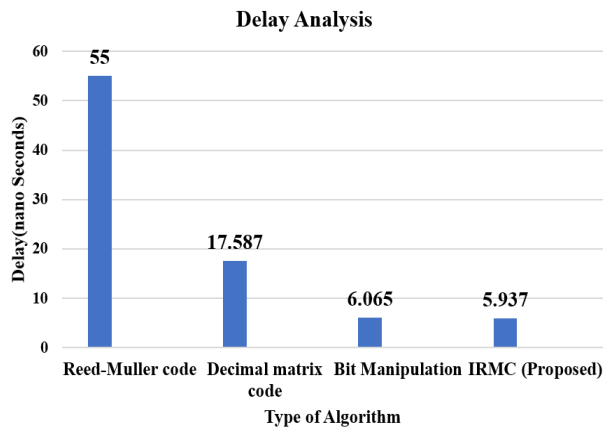


Fig. 12. Delay analysis of 32-bit input code

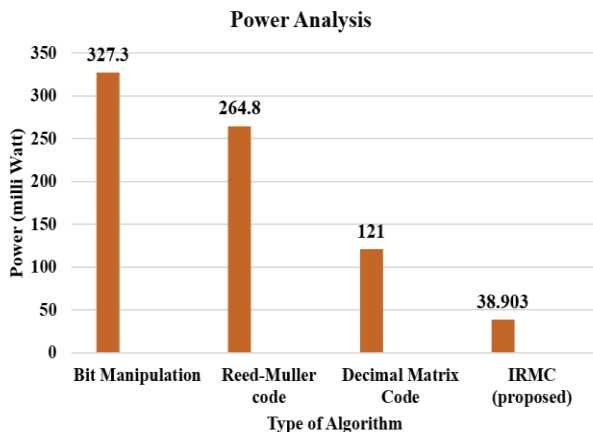


Fig. 13. Power analysis of 32-bit input code

As comes the next portion of comparison, for 64-bit input, IRMC produces 5.537ns as delay and 68.7mW as power.

Comparison table of delay & power for 64-bit input between IRMC and other algorithms:

Table 2 Delay and Power comparison between IRMC (proposed) and other existing algorithms

Methods	Power (mW)	Delay (nano seconds)
IRMC (proposed)	68.7mW	5.537ns
Decimal Matrix Code Algorithm^[10]	330.8mW	8.492ns
Bit Manipulation Algorithm^[10]	330.7mW	5.699ns

Graphical representation of 64-bit comparisons are described in figure 14 and figure 15:

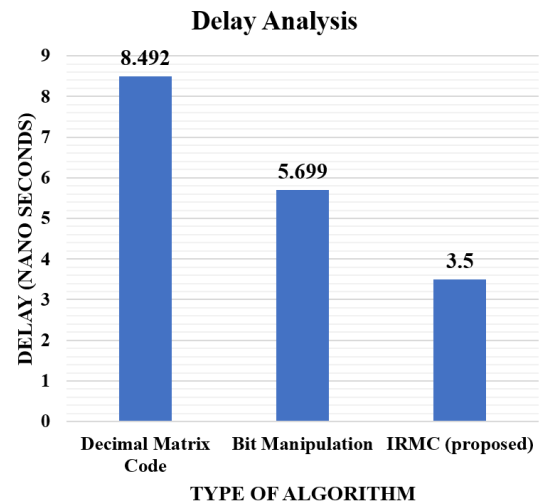


Fig. 14. Delay analysis of 64-bit input

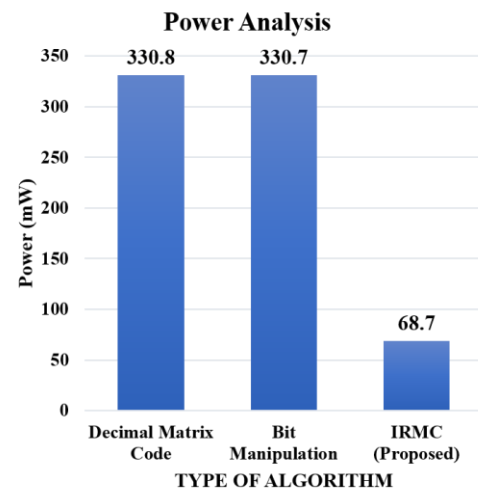


Fig. 15. Power analysis of 64-bit input

8. CONCLUSION & FUTURE SCOPE

An Improved Redundancy Matrix code algorithm provides a reliable and efficient approach to error detection and correction in VLSI. With the increasing demand for reliable digital communication systems, these algorithms are expected to play an important role in improving the performance and reliability of future systems such as integration with emerging technologies such as 5G, optimization of low-power devices, and implementation in various hardware electronic systems etc.

REFERENCES

- [1] M. Priya and M. M. Vijay, "Error Detection and Correction for SRAM Systems Using Improved Redundant Matrix Code," 2019 International Conference on Recent Advances in Energy-efficient Computing and Communication (ICRAECC), Nagercoil, India, 2019, pp. 1-8, doi: 10.1109/ICRAECC43874.2019.8995121.
- [2] R. Jenila, C. Supraja, C. K. Pappa and N. Dharani, "VLSI Implementation of Error Detection and Correction Codes for Space Engineering," 2021 Third International Conference on Intelligent Communication Technologies and Virtual Mobile Networks (ICICV), Tirunelveli, India, 2021, pp. 569-573, doi: 10.1109/ICICV50876.2021.9388477.
- [3] B. Liu, "Error-detecting/correcting-code-based self-checked/corrected/timed circuits," 2010 NASA/ESA Conference on Adaptive Hardware and Systems, Anaheim, CA, USA, 2010, pp. 66-72, doi: 10.1109/AHS.2010.5546217.
- [4] K. Kaviya Priya, Mr. M. Palaniappan, "Implementation of Error Detection and Correction Codes using VLSI", SSRG International Journal of Electronics and Communication Engineering (SSRG – IJECE) – Special Issue ICRTCRET Mar 2019.
- [5] Zhang, X.. (2017). VLSI Architectures for Modern Error-Correcting Codes. 10.1201/b18673.
- [6] P. Schulz, A. Traßl, A. N. Barreto and G. Fettweis, "Efficient and Reliable Wireless Communications via Multi-Connectivity Using Rateless Codes in Single- and Multi-User Scenarios," in IEEE Transactions on Wireless Communications, vol. 20, no. 9, pp. 5714-5729, Sept. 2021, doi: 10.1109/TWC.2021.3069669.
- [7] F. M. Hashemian, M. Shivaie, and A. Daneshtalab, "A Low-Complexity and High-Performance Error Correction Code for Non-Volatile Memories," IEEE Transactions on Very Large Scale Integration (VLSI) Systems, vol. 27, no. 10, pp. 2225-2236, Oct. 2019.
- [8] Y. Chen, H. Zhang, Y. Song, and D. Chen, "Low-Latency Polar Codes for HARQ Feedback in 5G Systems," IEEE Transactions on Vehicular Technology, vol. 68, no. 6, pp. 5641-5645, Jun. 2019.
- [9] E. Abinaya, Dr. D. Somasundareswari, S. Mathan Prasad (2015) 'Error Detection and Correction in 32-bit SRAM based FPGA memory using Decimal Matrix Code', International Journal of Science Technology & Management, Vol.04, No.1, pp. 2394-1529.
- [10] V. Bhanumathi, M. Sunandini (2019) 'VLSI Architecture for error detection and correction based on XOR against multiple cell upsets with reduced redundant bits', ICTACT Journal of Microelectronics, Vol.05, No.2, pp. 2395-1680.
- [11] V. Yatskiv, S. Kulyna, P. Bykovyy, T. Maksymyuk and A. Sachenko, "Method of Reliable Data Storage Based on Redundant Residue Number System," 2020 IEEE 5th International Symposium on Smart and Wireless Systems within the Conferences on Intelligent Data Acquisition and Advanced Computing Systems (IDAACS-SWS), Dortmund, Germany, 2020, pp. 1-4, doi: 10.1109/IDAACS-SWS50031.2020.9297052.
- [12] T. P. Le, P. H. Trong, and T. Q. Dang, "High-Throughput Syndrome-Based Error Correction Codes for NAND Flash Memory," IEEE Transactions on Very Large Scale Integration (VLSI) Systems, vol. 28, no. 9, pp. 2116-2120, Sep. 2020.
- [13] H. Heidarzadeh, S. E. Salami, and H. F. Rashvand, "Optimized Polar Codes for Beyond-5G Terahertz Communication Systems," IEEE Transactions on Communications, vol. 68, no. 2, pp. 1186-1195, Feb. 2020.
- [14] T. Wang, H. Liu, J. Lin, and C. Wang, "An Efficient and Low-Complexity Error Correction Scheme for Non-Volatile Memory," IEEE Transactions on Very Large Scale Integration (VLSI) Systems, vol. 28, no. 5, pp. 1139-1143, May 2020.
- [15] J. Zhou, Z. Liu, and Y. Li, "An Efficient Algorithm for Low-Density Parity-Check Codes Based on Min-Sum Message-Passing," IEEE Transactions on Communications, vol. 67, no. 6, pp. 4224-4234, Jun. 2019.
- [16] M. Xiao, M. Liu, H. Li, and Y. Li, "Polar Codes for Channel Coding in DNA Storage Systems," IEEE Transactions on Nanobioscience, vol. 19, no. 2, pp. 179-186, Jun. 2020.

ORIGINALITY REPORT

24%

SIMILARITY INDEX

18%

INTERNET SOURCES

19%

PUBLICATIONS

9%

STUDENT PAPERS

PRIMARY SOURCES

1

en.wikipedia.org

Internet Source

4%

2

api.openalex.org

Internet Source

3%

3

www.scilit.net

Internet Source

2%

4

M. Priya, M. M. Vijay. "Error Detection and Correction for SRAM Systems Using Improved Redundant Matrix Code", 2019 International Conference on Recent Advances in Energy-efficient Computing and Communication (ICRAECC), 2019

Publication

1%

5

dinosmos.blogs.upv.es

Internet Source

1%

6

researchid.co

Internet Source

1%

7

www.coursehero.com

Internet Source

1%

8	ictactjournals.in Internet Source	1 %
9	Helia Naeimi. "Fault Secure Encoder and Decoder for NanoMemory Applications", IEEE Transactions on Very Large Scale Integration (VLSI) Systems, 04/2009 Publication	1 %
10	baadalsg.inflibnet.ac.in Internet Source	1 %
11	Jonas Van Waes, Dries Vanoost, Jens Vankeirsbilck, Jonas Lannoo, Davy Pissoort, Jeroen Boydens. "Resilience of Error Correction Codes Against Harsh Electromagnetic Disturbances: Fault Elimination for Triplication-Based Error Correction Codes", IEEE Transactions on Electromagnetic Compatibility, 2020 Publication	1 %
12	ethesis.nitrkl.ac.in Internet Source	1 %
13	repositorio.ufc.br Internet Source	1 %
14	eiris.it Internet Source	1 %
15	www.researchgate.net Internet Source	1 %

16	Submitted to Government Engineering College, Thrissur Student Paper	1 %
17	www.fhwa.dot.gov Internet Source	<1 %
18	Communications in Computer and Information Science, 2011. Publication	<1 %
19	ijartet.com Internet Source	<1 %
20	annalsofrscb.ro Internet Source	<1 %
21	Marty, Paul F.. "Factors Influencing Error Recovery in Collections Databases: A Museum Case Study", The Library Quarterly, 2005. Publication	<1 %
22	digitalcommons.usf.edu Internet Source	<1 %
23	Jasim A. Ghaeb. "Integrity Assurance Technique Using Determinant Approach", 2009 Workshops at the Grid and Pervasive Computing Conference, 2009 Publication	<1 %
24	Submitted to Indiana University Student Paper	<1 %

25 Yan Sun. "Exploiting idle resources for reducing SER of microprocessor functional units", 2010 2nd International Conference on Computer Engineering and Technology, 04/2010
Publication

26 link.springer.com
Internet Source

27 Submitted to Visvesvaraya Technological University
Student Paper

28 tel.archives-ouvertes.fr
Internet Source

29 Andriy Semenov, Olena Semenova, Natalia Kryvinska, Vladimir Tromsyuk, Serhii Tsyruľnyk, Andrii Rudyk, Janusz Kacprzyk. "Advanced correlation method for bit position detection towards high accuracy data processing in industrial computer systems", Information Sciences, 2023
Publication

30 Meng Dong, Weitao Pan, Zhiliang Qiu, Xiaoxin Qi, Ling Zheng, Huan Liu. "A Universal, Low-Delay, SEC-DEC-TAEC Code for State Register Protection", IEEE Access, 2022
Publication

arxiv.org

31

Internet Source

<1 %

32

deepai.org

Internet Source

<1 %

33

www.intersil.com

Internet Source

<1 %

34

Allen, Phillip E.. "CMOS Analog Circuit Design",
Oxford University Press

Publication

<1 %

35

Maryam Ashouei, Soumendu Bhattacharya,
Abhijit Chatterjee. "Probabilistic
Compensation for Digital Filters Using
Pervasive Noise-Induced Operator Errors",
25th IEEE VLSI Test Symposium (VTS'07),
2007

Publication

<1 %

36

Yulun Zhou, Hongxia Liu, Qi Xiang, Chenyu
Yin. "High-Performance and Flexible Design
Scheme with ECC Protection in the Cache",
Micromachines, 2022

Publication

<1 %

37

hdl.handle.net

Internet Source

<1 %

38

"Proceedings of First International Conference
on Computational Electronics for Wireless

<1 %

39

Jonas Van Waes, Jens Vankeirsbilck, Jonas Lannoo, Davy Pissoort, Jeroen Boydens. "Effectiveness of Data Triplication in Harsh Electromagnetic Environments", 2018 International Symposium on Electromagnetic Compatibility (EMC EUROPE), 2018

Publication

<1 %

40

David Freitas, Jarbas Silveira, César Marcon, Lirida Naviner, João Mota. "OPCoSA: an Optimized Product Code for space applications", Integration, 2022

Publication

<1 %

41

Hamed Farbeh, Leila Delshadtehrani, Hyeonggyu Kim, Soontae Kim. "ECC-United Cache: Maximizing Efficiency of Error Detection/Correction Codes in Associative Cache Memories", IEEE Transactions on Computers, 2020

Publication

<1 %

42

T. Satyanarayana, V. A. Qureshi, G. Divya. "Design and implementation of error detection and correction system for semiconductor memory applications", 7th International Conference on Computing in Engineering & Technology (ICCET 2022), 2022

Publication

<1 %

APPENDIX - C
PROJECT TEAM

